QT 工业物联网开发实战
OpenOCD 烧录与 Modbus 通信
（V1.0）

深圳百问网科技有限公司

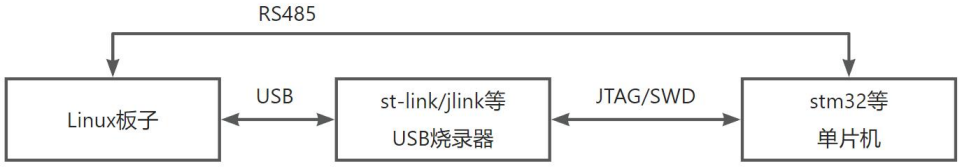
2025.4.28

第 1 章　课程、项目介绍

1.1 课程介绍

1.1.1 做个实用的项目

硬件框图如下：

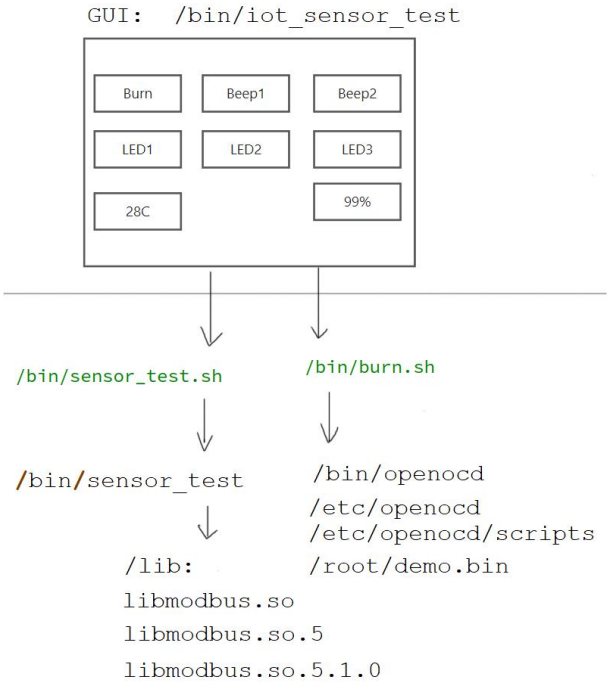


控制界面如下，可以点击“Burn”烧写单片机，点击其他按钮进行测试：



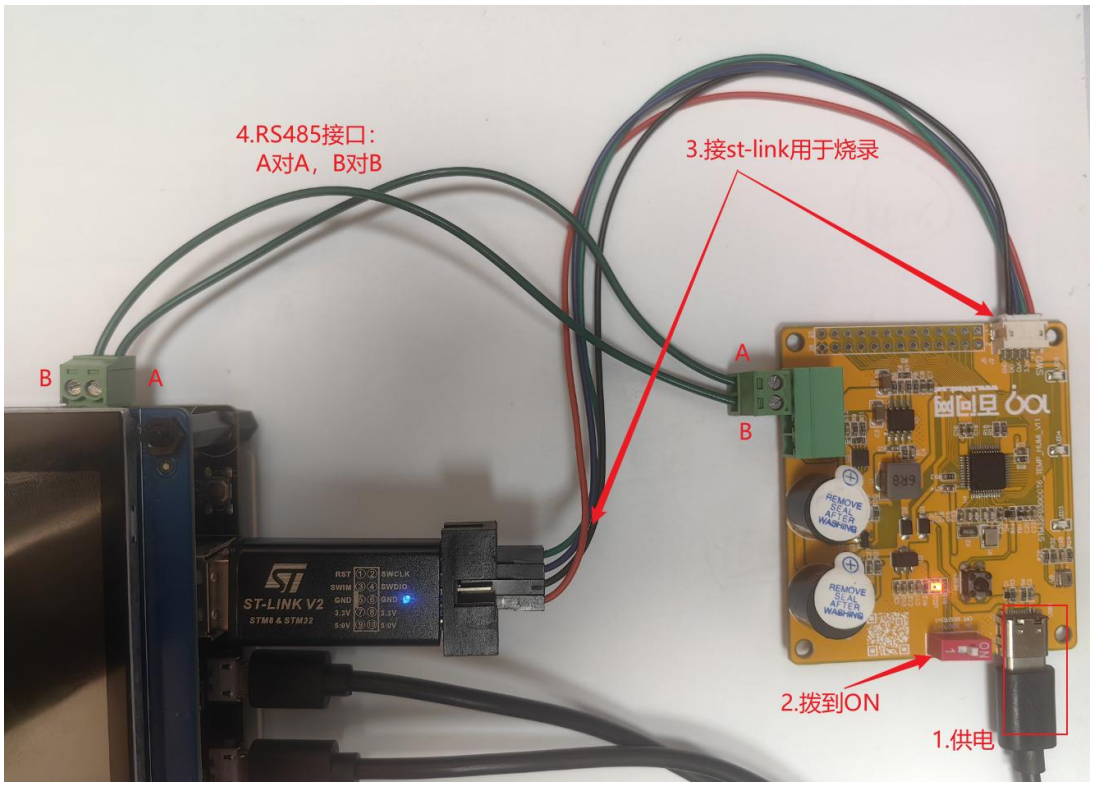
1.1.2 涉及内容

QT GUI 开发、使用开源软件烧写单片机、Modbus 的介绍与使用、镜像文件的制作。



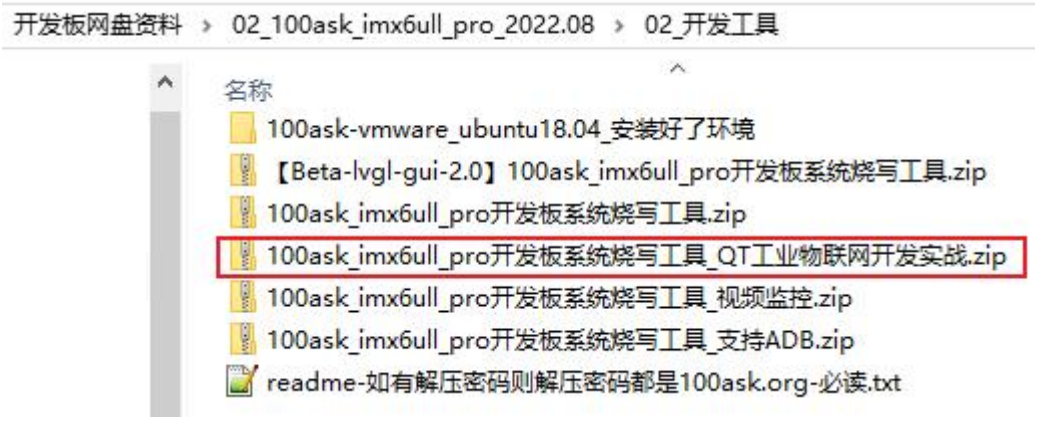
1.2 上机演示

1.2.1 接线



1.2.2 烧录

板子设置为 USB 启动，链接 USB OTG 线，使用下面工具烧录：



烧录后设置板子为 EMMC 启动，重新上电，即可在界面操作。

第 2 章　搭建开发环境

要下载的资料有 2 部分：
① 网盘资料：比较庞大、固定的资料，比如虚拟机、Ubuntu 镜像
② GIT 仓库：更新比较频繁的教程文档、源码：可以在 windows 下载，也可以在 Ubuntu 里面更新

2.1 下载网盘资料

打开百问网下载中心，在左侧找到你的开发板，下载网盘资料：



2.2 启动虚拟机、更新 GIT 仓库

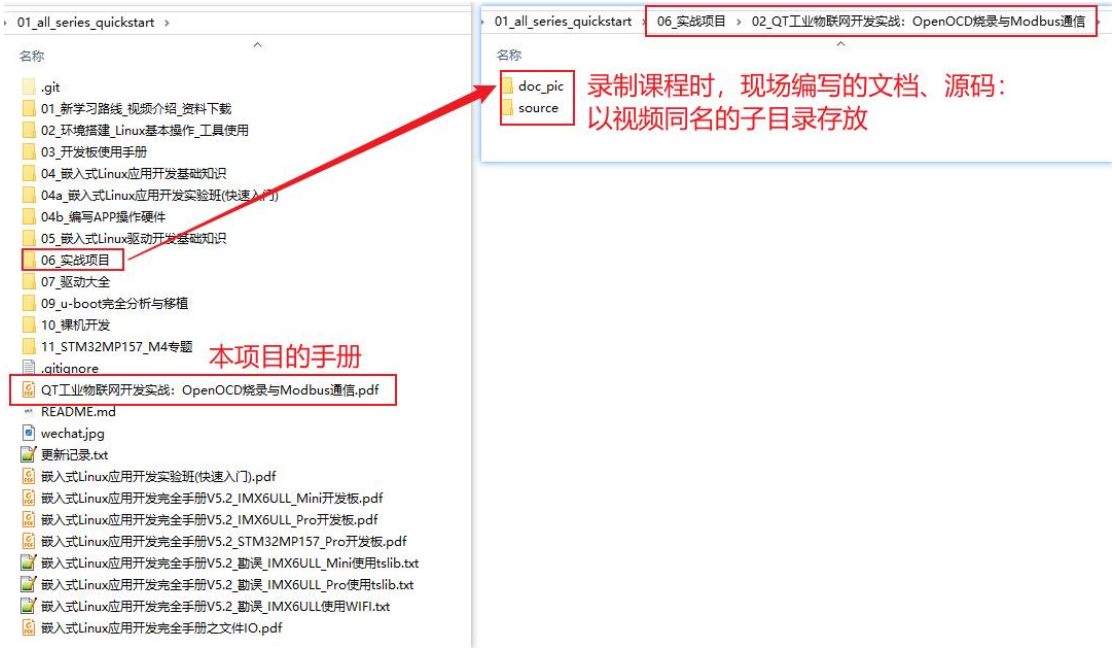
使用 vmware 打开如下虚拟机：



在 Ubuntu 里更新 GIT 仓库：

```
$ cd ~/Desktop/
$ cd 01_all_series_quickstart
$ git pull
```

GIT 仓库中资料摆放目录如下：

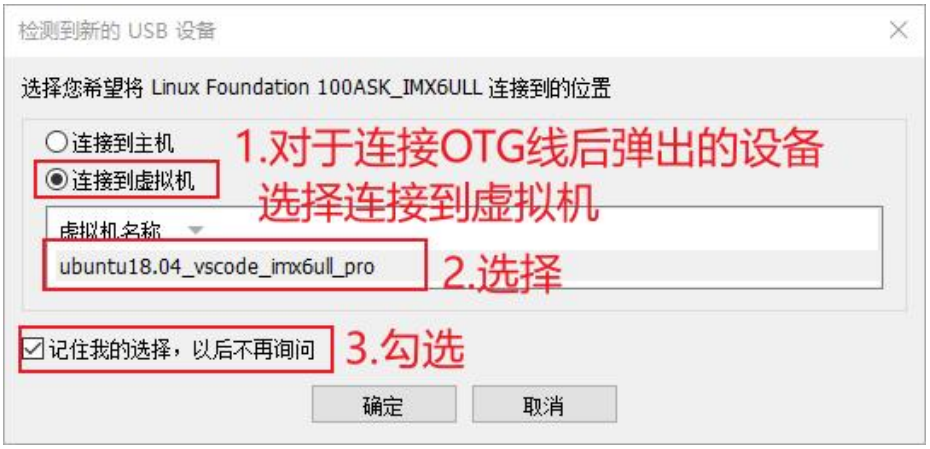


2.3 给开发板烧录支持 ADB 的系统

2.3.1 烧录系统

2.3.2 使用 adb

使用 USB 线连接开发板的 OTG 口和 PC，在 VMWare 里选择把新设备连接到 Ubuntu，如下：



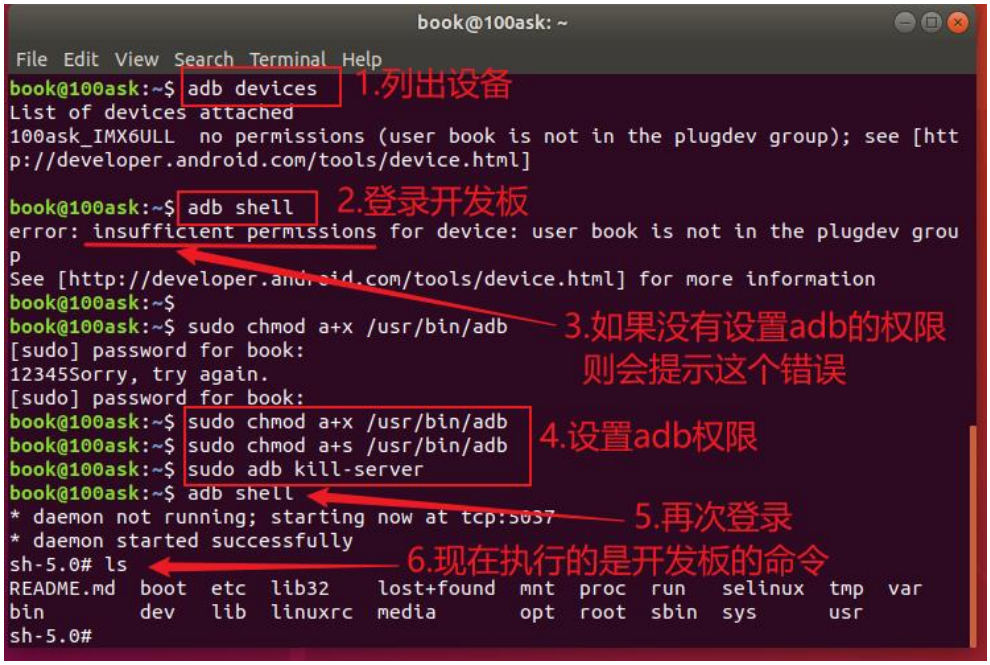
如果没有弹出上述界面，也可以如下操作：



然后就可以在 Ubuntu 上使用 adb 操作开发板了：

```
adb devices
adb push 1.txt /root       # 把Ubuntu的文件放到开发板的/root目录
adb pull /root/1.txt 2.tx # 把开发板的/root/1.txt下载并改名为2.txt
adb shell                  # 登录开发板
```

如下图所示：



2.4 体验第 1 个程序

编写 hello.c：

```
#include <stdio.h>
int main(int argc, char **argv)
{
    printf("hello, world!\n");
    return 0;
}
```

在 Ubuntu 上编译：

```
$ arm-buildroot-linux-gnueabihf-gcc -o hello hello.c
$ adb push hello  /root
```

然后就可以执行开发板下的“/root/hello”程序了。
如下图所示：



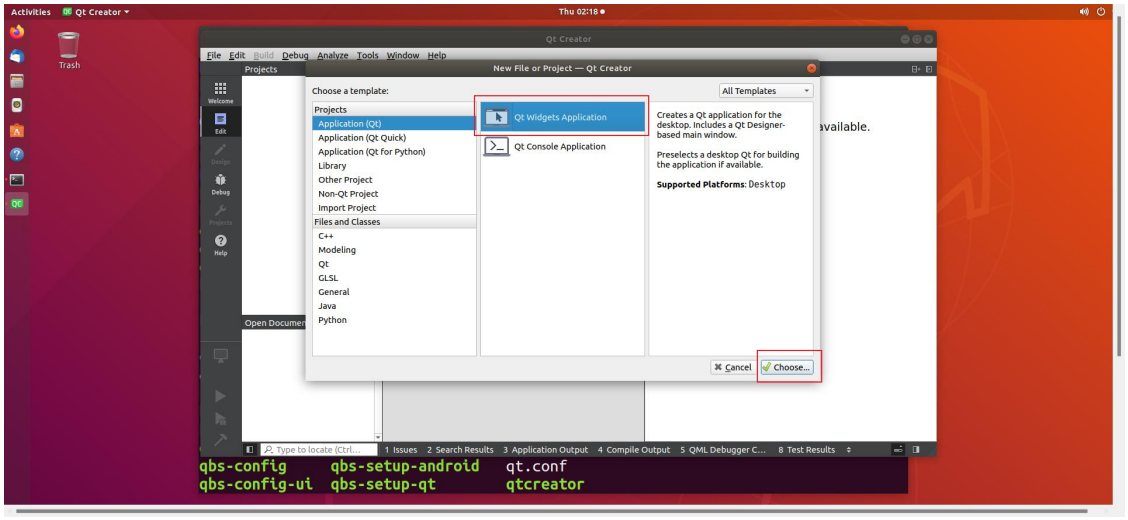
第 3 章　QT GUI 程序开发

3.1 编写 QT 程序

第 1 步创建新项目。

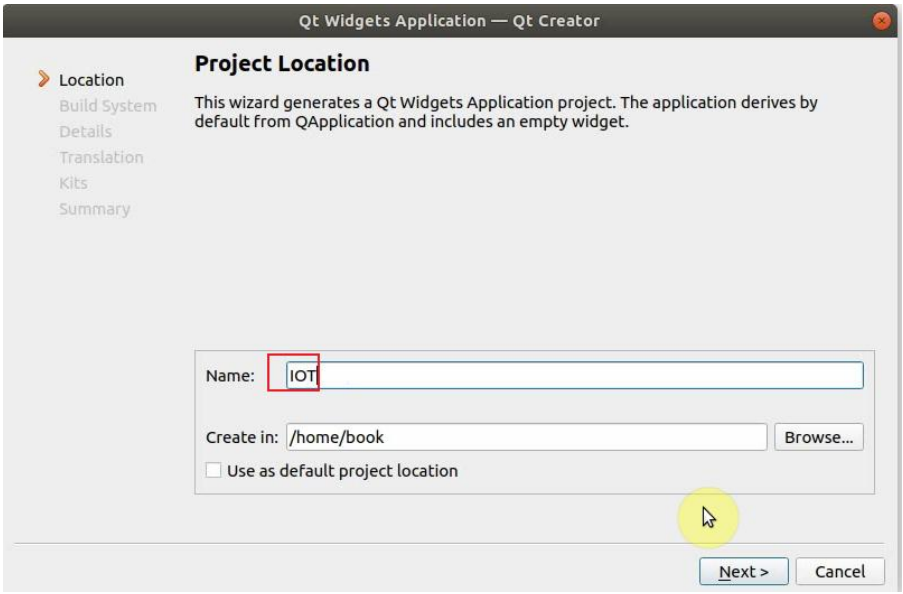
运行 QtCreator 后，在菜单栏选择 File -> New File or Project，选择项目类型。

在打开的对话框中，依次选择 Application -> Qt Widgets Application ，点击 Choose... ，如下图所示：

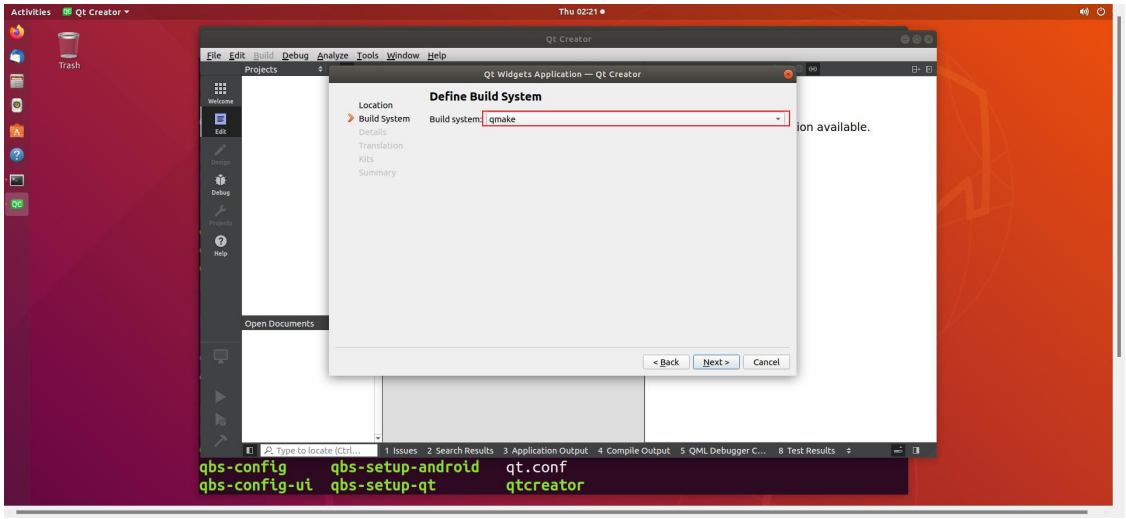


第 2 步：输入项目名字、设置保存位置。

在弹出的 Qt Widgets Application 对话框中，设置项目名称为“LED_and_TempHumi”,Create in 一栏填写项目的存储路径（点击右边的 Brouse 按钮创建这个目录），点击 Next。如下：

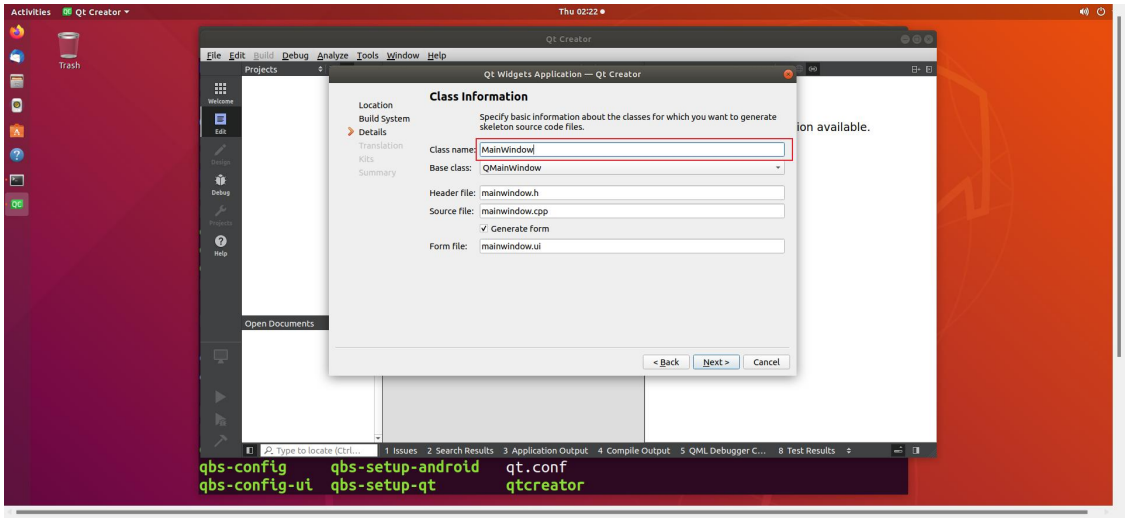


第 3 步 设置构建系统，然后点击 Next 进入下一步，如下：

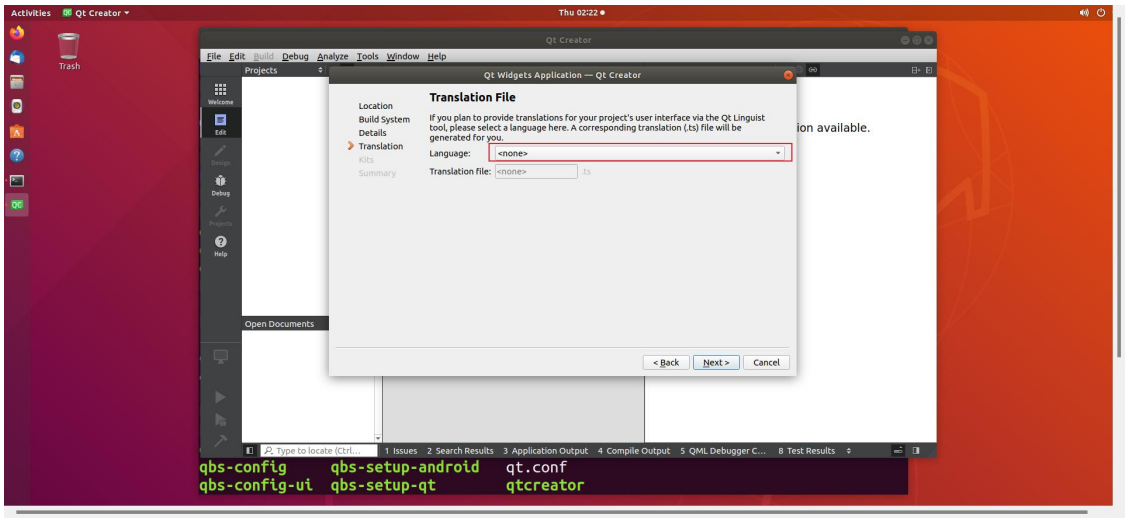


第 4 步 选择基类：

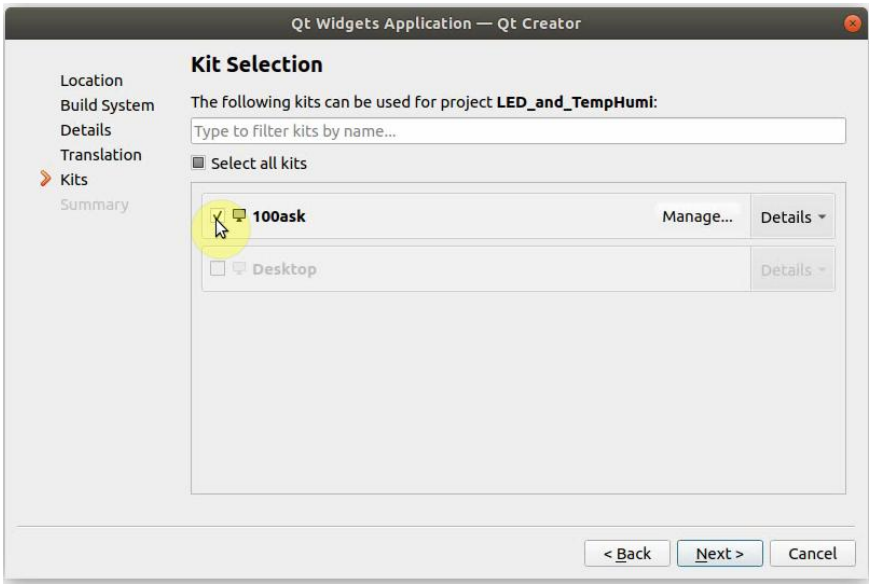
当前的的应用继承自哪种 Widget,默认选择 QMainWindow,然后点击 Next 进入下一步。



第 5 步 默认 none，继续点击 next：

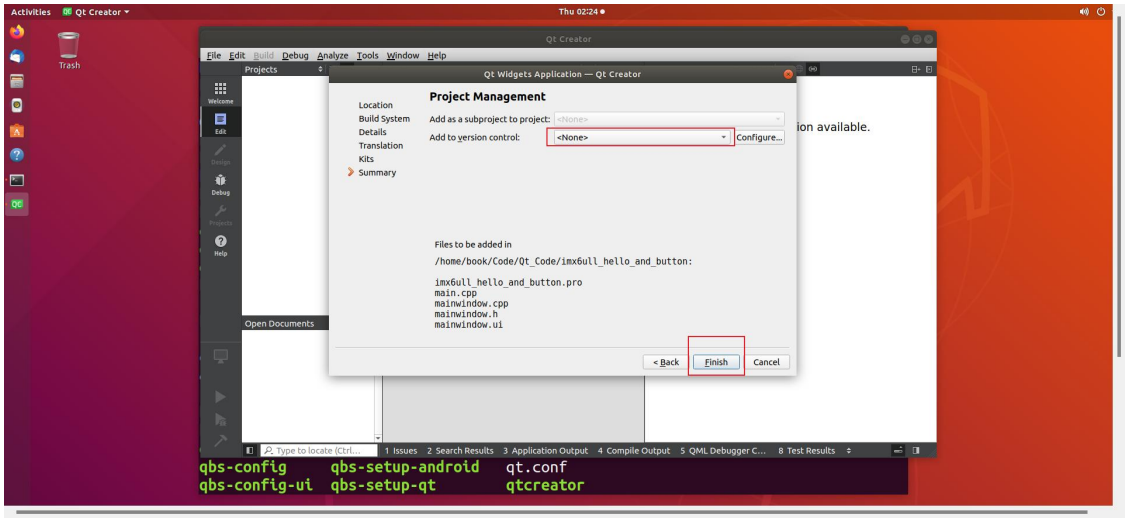


第 6 步 选择之前添加好的 Kits(100ask)，继续点击 next：



第 7 步 完成项目创建：

以上信息填写完后，点击 Finish，完成项目的创建。



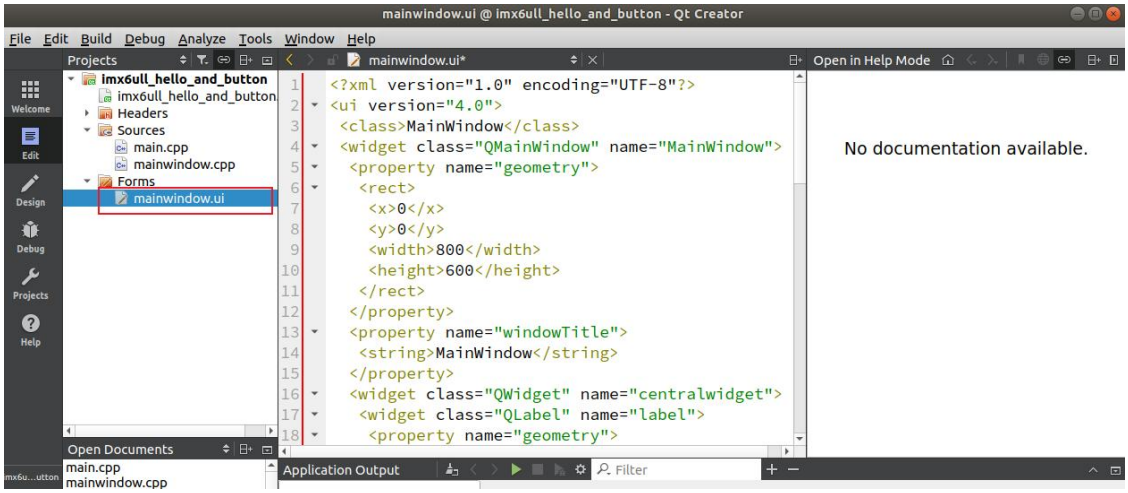
3.2 编译 Qt 程序

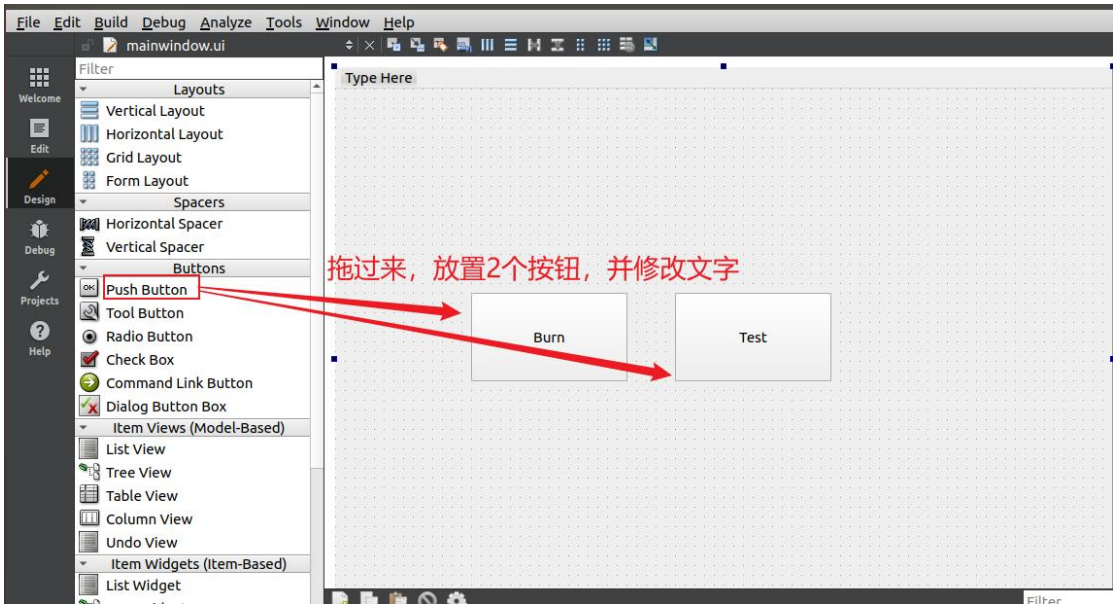
第 1 步 修改界面：

双击左侧的 Forms 里的 mainwindow.ui 文件，打开 Design 视图。

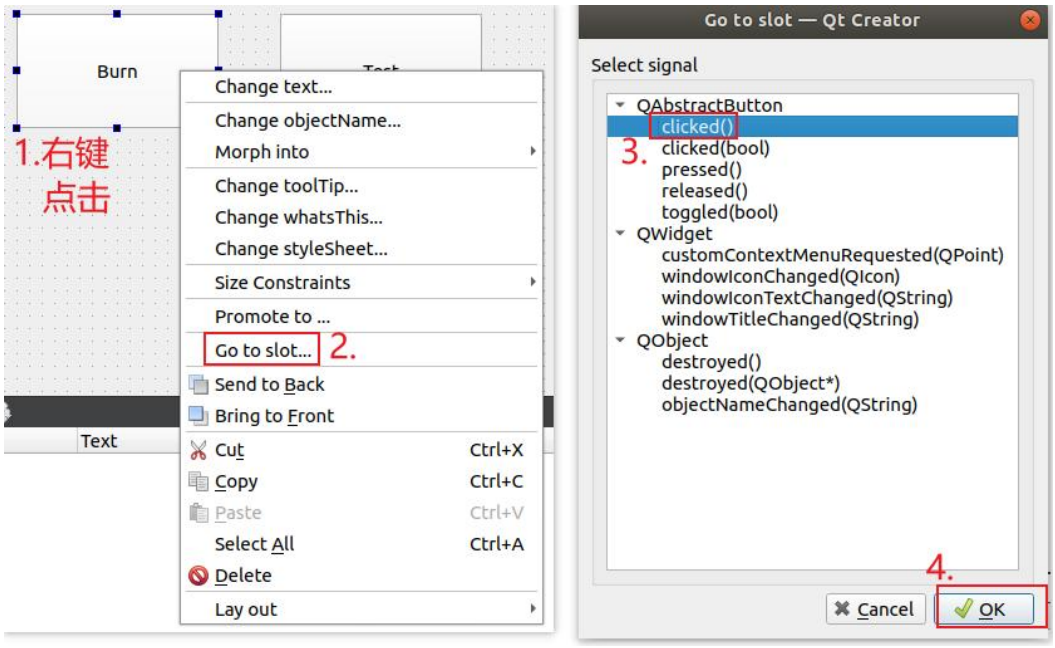
然后如下图所示，从左侧 Display Widgets 栏目下，拖动两个 PushPutton 到中间的区域。

最后双击中间区域的 Label 还有 Push Button 修改内容如下图：





第 2 步 给 Burn 添加对应函数（使用同样的方法给 Test 按钮添加函数）：



第 3 步，添加代码：

```
#include "ui_mainwindow.h"
#include <QDebug>

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);
}

MainWindow::~MainWindow()
{
    delete ui;
}


void MainWindow::on_pushButton_clicked()
{
    /* burn button */
    qDebug() << "Burn Button clicked!";
}

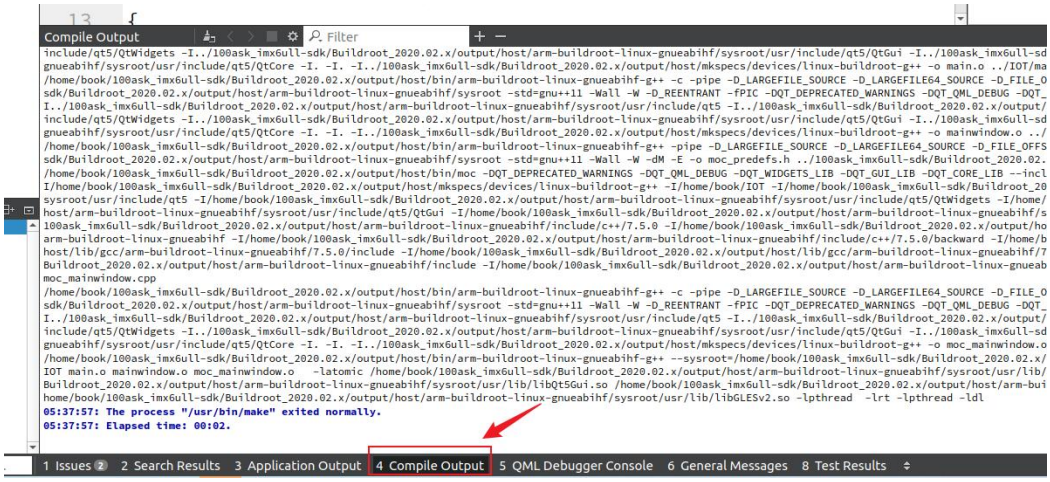
void MainWindow::on_pushButton_2_clicked()
{
    /* test button */
    qDebug() << "Test Button clicked!";
}
```

第 4 步 构建：

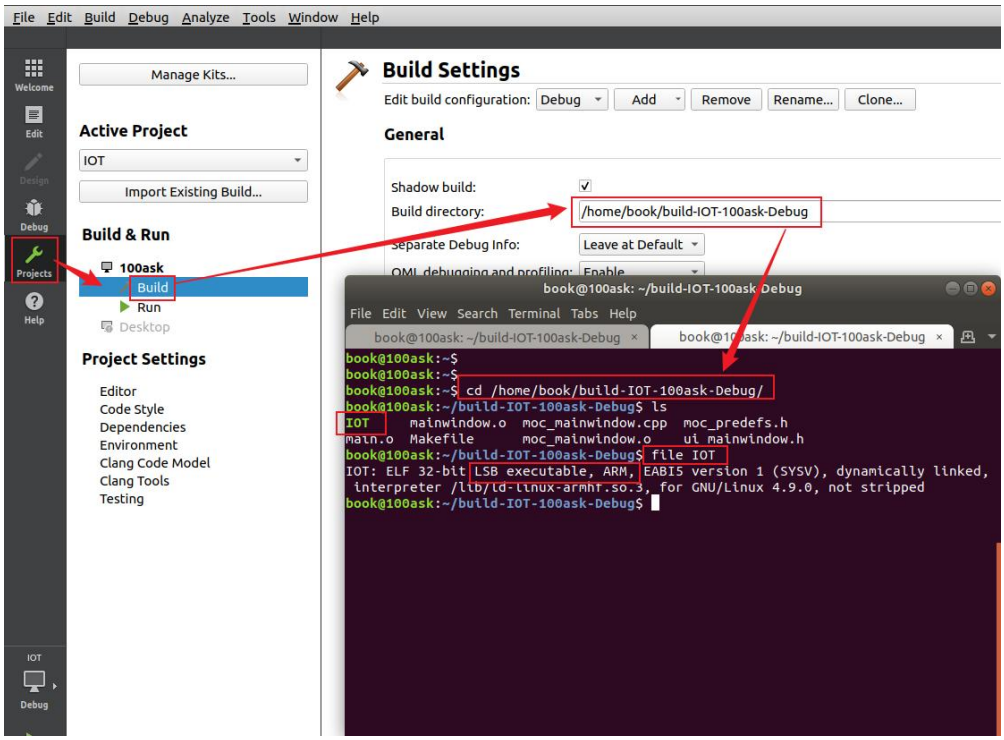
点击菜单栏 Build -> Build Project IOT，开始编译、构建项目。

在构建过程中，会在左下侧是“Compile Output”窗口打印构建信息。如果有错误，请

根据提示出错信息修改，然后重新构建。



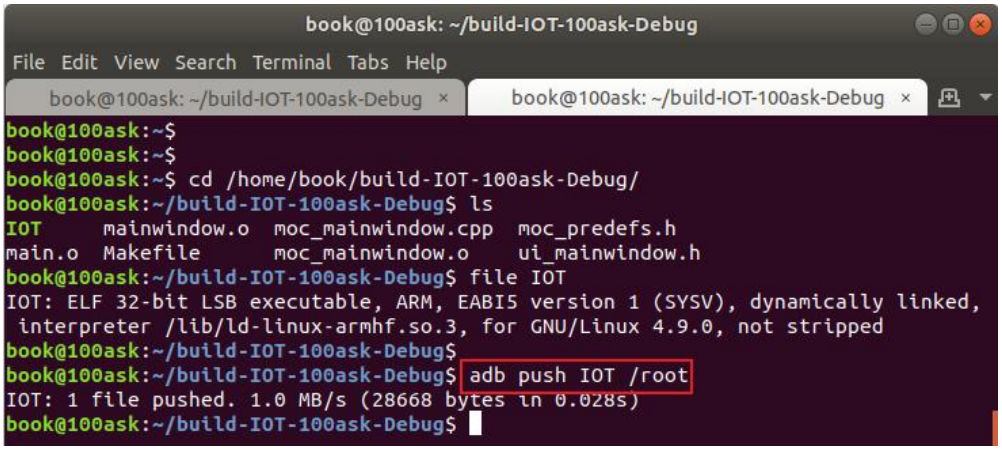
第 5 步 查看构建结果：项目构建成功后，编译好的二进制文件存放在刚刚设置的目录中，可以使用 file 命令查看该 APP 是否被编译为 ARM 架构：



3.3 在开发板上运行 Qt 程序

3.3.1 手工运行

使用 ADB 把可执行程序 LED_and_TempHumi 放到到开发板上：



在开发板上执行如下命令：

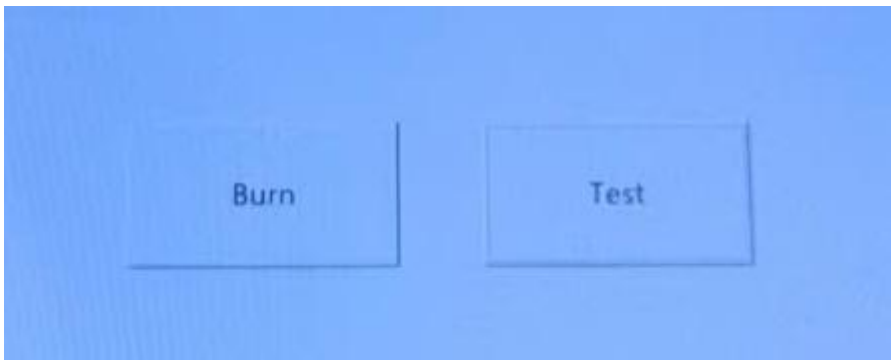
```
// 1. 关闭原来的GUI
mv /etc/init.d/*lvgl* /root
mv /etc/init.d/*hmi* /root
reboot

// 2. 禁止LCD黑屏
echo -e "\033[9;0]" > /dev/tty0

// 3. 设置QT运行环境
export QT_QPA_GENERIC_PLUGINS=tslib:/dev/input/event1
export QT_QPA_PLATFORM=linuxfb:fb=/dev/fb0
export QT_QPA_FONTDIR=/usr/lib/fonts/

// 4. 运行QT程序
/root/IOT
```

在屏幕上可以看到如下界面：



点击Burn、Test按钮，可以在控制台看到如下输出信息：

```
book@100ask:~/build-IOT-100ask-Debug$ adb shell
sh-5.0#
sh-5.0#
sh-5.0# echo -e "\033[9;0]" > /dev/tty0
sh-5.0# export QT_QPA_GENERIC_PLUGINS=tslib:/dev/input/event1
sh-5.0# export QT_QPA_PLATFORM=linuxfb:fb=/dev/fb0
sh-5.0# export QT_QPA_FONTDIR=/usr/lib/fonts/
sh-5.0#
sh-5.0# /root/IOT
QStandardPaths: XDG_RUNTIME_DIR not set, defaulting to '/tmp/runtime-root'
qt.qpa.input: xkbcommon not available, not performing key mapping
Burn Button clicked!
Test Button clicked!
```

3.3.2 自动运行

创建一个文件：/etc/init.d/S99myqt，内容如下：

```
#!/bin/sh

start() {
echo -e "\033[9;0]" > /dev/tty0
export QT_QPA_GENERIC_PLUGINS=tslib:/dev/input/event1
export QT_QPA_PLATFORM=linuxfb:fb=/dev/fb0
export QT_QPA_FONTDIR=/usr/lib/fonts/

psplash-write "PROGRESS 95"
psplash-write "QUIT"
/root/IOT  &
}

stop() {
    killall  IOT
}

case "$1" in
    start)
        start
        ;;
    stop)
        stop
        ;;
    *)
        echo "Usage: $0 {start| stop|restart}"
        exit 1
esac

exit $?
```

以后开发板启动后即可自动运行QT程序。

第 4 章　使用 openocd 烧写传感器

4.1 技术方案

4.1.1 硬件连接



4.1.2 软件方案

本项目在Linux板子上使用Openocd软件，通过st-link烧写传感器。

OpenOCD（Open On-Chip Debugger）是一款功能强大的开源工具，主要用于嵌入式系统的片上编程和调试。

它有如下功能：

① 调试功能 ：支持多种调试协议，如 JTAG、SWD、BDM 等，能够与目标设备通信，进行读写内存、设置断点、单步执行代码等操作，帮助开发者定位和解决问题。

② 编程功能 ：可将程序代码烧录到目标设备的闪存中，方便进行固件更新和程序加载，支持多种类型的 Flash 存储器，如 CFI、NAND 和内部 Flash 等。

③ 丰富的接口与集成性 ：支持 Telnet、TCL 和 GDB 等多种网络接口，可作为 GDB 的远程目标，实现基于源码级别的调试，与 GDB 等调试器无缝集成，还支持与 STM32CubeIDE、CLion、Segger Studio 和 PlatformIO 等多种开发工具集成。

④ 脚本支持与自动化 ：集成 TCL 解释器，允许用户编写脚本来自动化调试流程，提高工作效率，还可使用 Tcl 脚本来自定义调试环境。

⑤ 多架构与多硬件支持 ：支持多种处理器架构，如 ARM、MIPS、RISC-V 等，以及大量的具体芯片型号，还支持广泛的 JTAG 适配器，如 J-Link、ST-LINK、USB-Blaster 等。

⑥ 跨平台支持 ：支持 Windows、Linux、macOS 等多种操作系统，方便开发者在不同开发环境中使用。

应用场景为：

① 嵌入式开发调试 ：在嵌入式系统开发过程中，帮助开发者调试代码，定位问题，如在开发基于 ARM、MIPS 等架构的微控制器的应用时，可连接目标设备进行实时调试。

② 固件烧录与更新 ：将开发好的固件烧录到嵌入式设备的 Flash 中，实现程序的加载和更新，适用于各种需要频繁更新固件的开发场景。

③ 硬件测试与诊断 ：用于诊断目标设备的硬件问题，如内存错误、时钟问题等，还可通过对设备的边界扫描操作，检测硬件电路的连通性、焊点质量等。

④ FPGA/CPLD 编程与调试 ：支持自动化边界扫描和 FPGA/CPLD 编程，方便进行硬件配置和测试，可在 FPGA 项目中下载配置文件，对 FPGA 进行调试和测试。

⑤ 远程调试 ：通过网络连接进行远程调试，方便开发者对远端的设备进行调试和配置，适用于分布式开发团队或对远程设备进行维护的场景

4.2 Openocd 的编译与使用

4.2.1 交叉编译

1. 编译 jimtcl

命令如下：

```
unzip jimtcl-master.zip
cd jimtcl-master/
ls
./configure --host=arm-buildroot-linux-gnueabihf --prefix=$PWD/tmp
make
make install
ls tmp/
```

2. 编译 libusb

命令如下：

```
unzip libusb-1.0.28.zip
cd libusb-1.0.28/
./bootstrap.sh
./configure --host=arm-buildroot-linux-gnueabihf --prefix=$PWD/tmp
make
make install
```

3. 编译 openocd

命令如下：

```
unzip openocd-master.zip
cd  unzip openocd-master
./bootstrap

JIMTCL_CFLAGS="-I /home/book/openocd/jimtcl-master/tmp/include"  JIMTCL_LIBS="-
L/home/book/openocd/jimtcl-master/tmp/lib  /home/book/openocd/jimtcl-
master/tmp/lib/libjim.a"   LIBUSB1_CFLAGS="-I /home/book/openocd/libusb-
1.0.28/tmp/include/libusb-1.0/"  LIBUSB1_LIBS="-L/home/book/openocd/libusb-1.0.28/tmp/lib -
lusb-1.0" ./configure --host=arm-buildroot-linux-gnueabihf  --enable-stlink --
prefix=$PWD/tmp

make  // 会在最后链接时出错，手工执行下面的链接命令

arm-buildroot-linux-gnueabihf-gcc -Wall -Wstrict-prototypes -Wformat-security -Wshadow -
Wextra -Wno-unused-parameter -Wbad-function-cast -Wcast-align -Wredundant-decls -Wpointer-
arith -Wundef -Werror -g -O2 -o src/openocd src/main.o  src/.libs/libopenocd.a -
L/home/book/openocd/libusb-1.0.28/tmp/lib /home/book/openocd/libusb-1.0.28/tmp/lib/libusb-
```

```
1.0.so   -lpthread -lm -L/home/book/openocd/jimtcl-master/tmp/lib /home/book/openocd/jimtcl-master/tmp/lib/libjim.a -lutil -ldl -pthread -Wl,-rpath -Wl,/home/book/openocd/libusb-1.0.28/tmp/lib  -lssl -lz -lcrypto

make install
```

4.2.2 烧写单片机

1．上传文件到单板

执行如下命令：

```
cd  /home/book/openocd/openocd-master/tmp/bin
adb  push  openocd   /bin
cd  /home/book/openocd/openocd-master/tmp/share
tar czf openocd.tar.gz openocd
adb push openocd.tar.gz /etc
```

2．在开发板上的操作

```
cd  /etc
gunzip  openocd.tar.gz
tar  xf  openocd.tar
ls  /etc/openocd
```

烧写命令为：

```
cd /etc/openocd/scripts

openocd -f interface/stlink.cfg -f target/stm32f0x.cfg -c init -c halt -c "flash write_image erase /root/leds.bin 0x08000000" -c reset -c shutdown
```

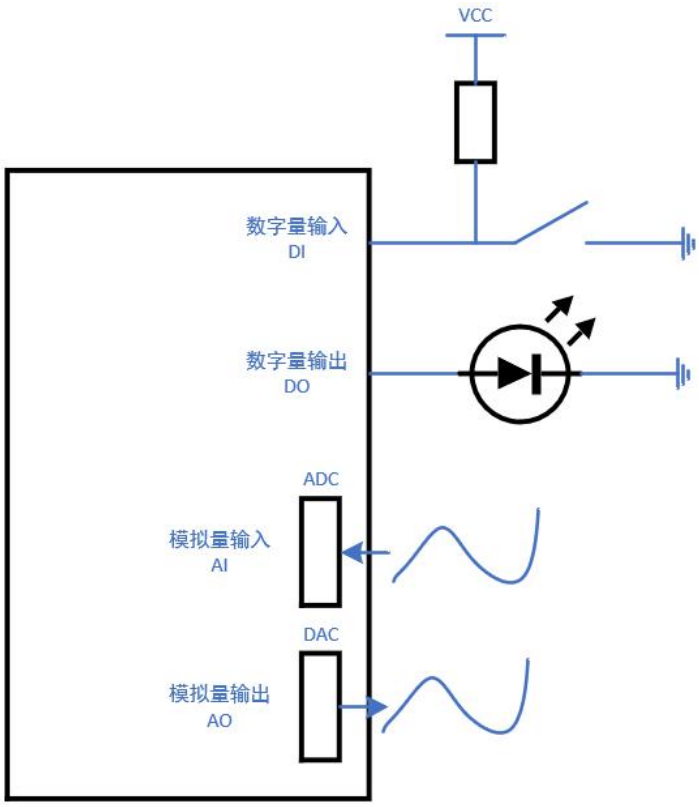
第 5 章　Modbus RTU 协议与编程

5.1 学习 Modbus 的快速方法

5.1.1 寄存器速记

作为初学者，你阅读 Modbus 协议时会发现它的概念别扭、重复、不易区分，比如线圈状态（Coil Status）、离散输入状态（Discrete Input Status）、保持寄存器（Holding Register）、输入寄存器（Input Register）。

回到事情的本质，在工业控制 PLC 领域，涉及数字信号的输入、输出，模拟信号的输入、输出，如下图所示：



对于软件开发而言：

① 想得到按键输入状态时，读取到的是一位数据；

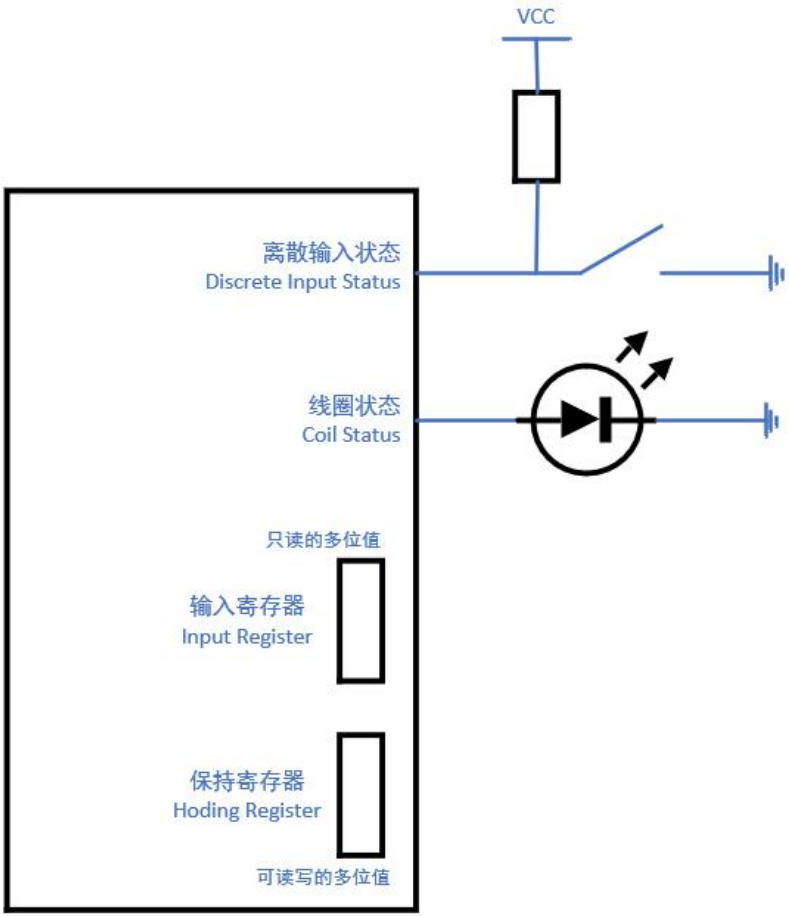
② 想控制 LED 时，需要输出一位数据，想读取 LED 当前状态时，也可以读取到一位数据

③ 想读取模拟信号时，读取到的是多位数据，比如 16 位数据

④ 想输出模拟信号时，写入的是多位数据，比如 16 位数据；也可以读取“模拟量输出”的当前值。

在上图中，“数字量输入 DI”是只读的，“数字量输出 DO”是可读可写的，“模拟量输入 AI”是只读的，“模拟量输出 AO”是可读可写的。

上图里的“模拟量输入 AI”、“模拟量输出 AO”都表示“多位数值”，这些“多位数值”无需局限于只表示“模拟量”，也可以表示“多位数字量”。把 AI、AO 的含义扩展后，如下图所示：



对于软件开发而言：

① 想得到按键输入状态时，读取到的是一位数据；

② 想控制 LED 时，需要输出一位数据，想读取 LED 当前状态时，也可以读取到一位数据

③ 想读取参数时，读取到的“输入寄存器”，得到多位数据，比如 16 位数据

④ 想设置参数时，写的是“保存寄存器”，写入的是多位数据，比如 16 位数据；也可以读“保存寄存器”

在电子系统里，无论是单 bit 的数值、多 bit 的数值，都是保存在寄存器里。根据上图，这些寄存器可以分为 4 类：

寄存器种类	说明	与 PLC 类比	举例说明
线圈状态 (Coil Status)	输出端口。可设定端口输出状态，也可以读取该位的输出状态。可分为两种不同的执行状态，列如保持型或边沿触发型	DO(数字量输出)	电磁阀输出、MOSFEF 输出、LED 显示等
离散输入状态(Discrete Input Status)	输入端口。通过外部设定改变输入状态，可读但不可以写	DI(数字量输入)	拨码开关、接近开关等
保持寄存器(Holding Register)	输出参数或保持参数，控制器运行时被设定的某些参数，可读可写	AO(模拟量输出)	模拟量输出设定值，PID 运行参数，变量阀输出大小，传感器报警上限下限
输入寄存器（Input Register）	输入参数。控制器运行时从外部设备获得的参数，但可读不可写	AI(模拟量输入)	模拟量输入

在 Modbus 中，多位操作时都是 16 位（2bytes）的，总结如下：

① bit 操作涉及的寄存器有 2 类：线圈状态（可读可写）、离散输入状态（只读）

② 16bit 操作的寄存器有 2 类：保存寄存器（可读可写）、输入寄存器（只读）

一个设备里，可能有多个“线圈状态”、多个“离散输入状态”、多个“保存寄存器”、多个“输入寄存器”。怎么分辨某类寄存器中的某一个？它们有“寄存器地址”，如下图所示：

寄存器种类	PLC 寄存器地址范围	Modbus 寄存器地址范围	简称	读写状态
线圈状态	00001~09999	0000H~FFFFH	0x	可读可写
离散输入状态	10001~19999	0000H~FFFFH	1x	只读
保持寄存器	40001~49999	0000H~FFFFH	4x	可读可写
输入寄存器	30001~39999	0000H~FFFFH	3x	只读

在上表中，“线圈状态”的寄存器 N、“离散输入状态”的寄存器 N，是两个不同的寄存器。

简单记忆方法：“

① 偶数类的寄存器”是可读可写的，比如“0x”和“4x”；

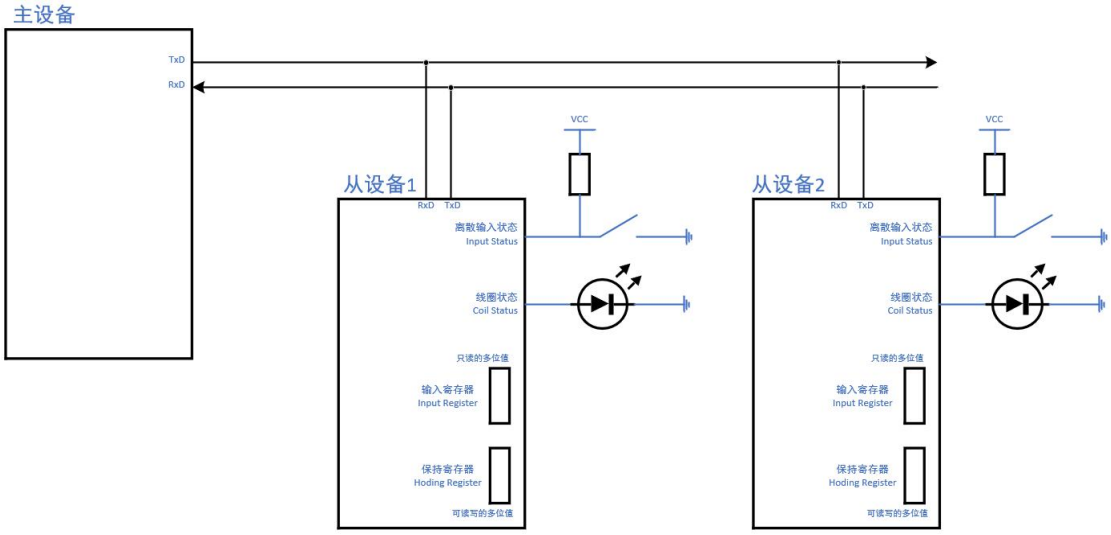
② “奇数类的寄存器”是只读的，比如“1x”和“3x”；

③ “0x”和“1x”是 bit 寄存器；

④ “3x”和“4x”是 16bit 寄存器。

5.1.2 协议速记

Modbus 是一主多从的协议，如下图所示：



主控发出的数据里，必定含有如下信息：

① 设备地址：你要访问从设备 1，还是访问从设备 2
② 访问哪类寄存器，是读还是写，只访问 1 个寄存器，还是多个寄存器：这被称为功能码
③ 起始寄存器地址、寄存器数量：这在数据里定义
④ 为了保证数据传输的可靠，还附带有 CRC 检验码

以 Modbus RTU 协议为例，主控发出的数据包格式如下：

起始间隙	设备地址	功能代码	数据	CRC校验	停止间隙
≥3.5个字符	8位	8位	Nx8位	16位	≥3.5个字符

功能代码有哪些？常用的功能码如下：

① 读线圈状态（01）
② 读离散输入状态（02）
③ 写单个线圈（05）、写多个线圈（15）
④ 读保持寄存器（03）
⑤ 读输入寄存器（04）
⑥ 写单个保存寄存器（06）、写多个保存寄存器（16）

数据的格式，由功能代码确定。以“读线圈状态”为例，主控发出的请求、从设备返回的响应包，或者从设备返回的错误包，格式如下：

Request

Function code	1 Byte	**0x01**
Starting Address	2 Bytes	0x0000 to 0xFFFF
Quantity of coils	2 Bytes	1 to 2000 (0x7D0)

Response

Function code	1 Byte	**0x01**
Byte count	1 Byte	**N***
Coil Status	**n** Byte	n = N or N+1

***N** = Quantity of Outputs / 8, if the remainder is different of 0 ⇒ N = N+1

Error

Function code	1 Byte	**Function code + 0x80**
Exception code	1 Byte	01 or 02 or 03 or 04

上图中，寄存器起始地址（“Starting Address”）是 16 位的，先传输高字节，再传输

低字节。线圈数量（“Quantiti of coils”）也是 16 位的，先传输高字节，再传输低字节。

响应包回复多少个数据呢（上图中 N 为多少）？N = Quantiti of coils / 8，如果余数不等于 0，则 N 再加 1。比如 Quantiti of coils=9，则返回 2 个字节。

在《Modbus_Application_Protocol_V1_1b3.pdf》中，列出了如下功能表。根据次表，在结合《5.5 Moubus 功能码详解》的示例，就可以对 Modbus RTU 协议有很好的理解了。

				Function Codes			
				code	*Sub code*	*(hex)*	*Section*
Data Access	**Bit access**	Physical Discrete Inputs	Read Discrete Inputs	**02**		**02**	6.2
		Internal Bits Or Physical coils	Read Coils	**01**		**01**	6.1
			Write Single Coil	**05**		**05**	6.5
			Write Multiple Coils	**15**		**0F**	6.11
	16 bits access	Physical Input Registers	Read Input Register	**04**		**04**	6.4
		Internal Registers Or Physical Output Registers	Read Holding Registers	**03**		**03**	6.3
			Write Single Register	**06**		**06**	6.6
			Write Multiple Registers	**16**		**10**	6.12
			Read/Write Multiple Registers	**23**		**17**	6.17
			Mask Write Register	**22**		**16**	6.16
			Read FIFO queue	**24**		**18**	6.18
	File record access		Read File record	**20**		**14**	6.14
			Write File record	**21**		**15**	6.15
Diagnostics			Read Exception status	**07**		**07**	6.7
			Diagnostic	**08**	**00-18,20**	**08**	6.8
			Get Com event counter	**11**		**OB**	6.9
			Get Com Event Log	**12**		**0C**	6.10
			Report Server ID	**17**		**11**	6.13
			Read device Identification	**43**	**14**	**2B**	6.21
Other			Encapsulated Interface Transport	**43**	**13,14**	**2B**	6.19
			CANopen General Reference	**43**	**13**	**2B**	6.20

5.2 编程读写传感器

5.2.1 技术方案

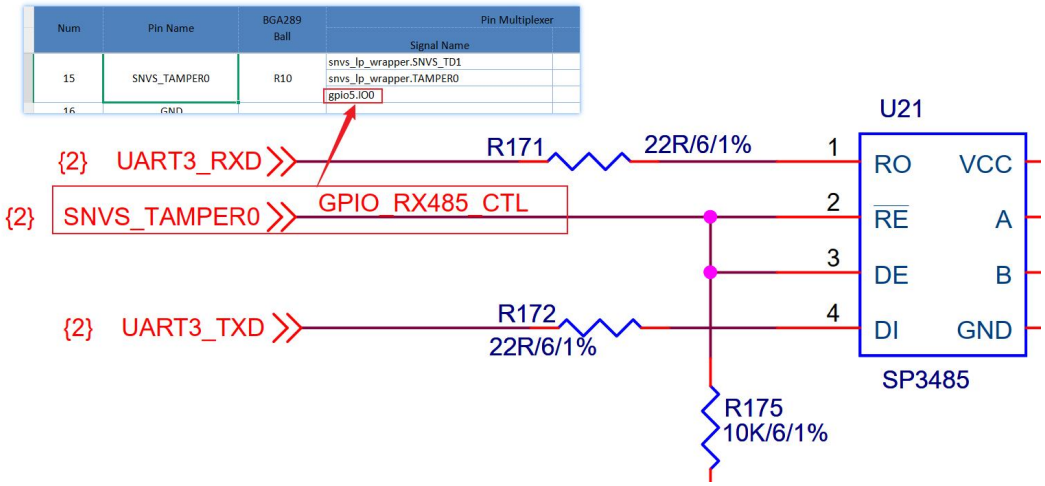
向 AI 工具提问：在 linux arm 平台上访问 modbus rtu 的方案。
根据它的回答，我们选择使用 libmodbus。

再次提问：我使用 C 语言与 libmodbus 库来访问 modbus rtu，实现步骤是怎样的。
它的回答是使用 apt 安装 libmodbus，但是我们的板子没有 apt 命令。

得到答案：
① 交叉编译 libmodbus
② 编写程序

最后的问题：我的 RS485 接口需要使用 GPIO 控制收发，在哪里、怎么修改程序

```
// 注册回调函数
modbus_set_pre_transmission_callback(ctx, pre_transmission_callback);
modbus_set_post_transmission_callback(ctx, post_transmission_callback);
```



5.2.2 交叉编译 libmodbus

在 Ubuntu 上执行如下命令：

```
tar xzf libmodbus-3.1.10.tar.gz
cd libmodbus-3.1.10/
./configure --host=arm-buildroot-linux-gnueabihf --prefix=$PWD/tmp
make
make install
cd tmp/
ls
```

5.2.3 编写程序

1．传感器点表

对于 Modbus 设备，怎么访问它们？它们的寄存器分别有什么功能？这在“点表”里描述，每个寄存器被称为一个“点”。

百问网的温湿度传感器的点表如下：

设备地址	寄存器地址	寄存器类别	用途	描述
03H	0000H	DO	控制蜂鸣器 1	1-响
	0001H	DO	控制蜂鸣器 2	1-响
	0002H	DO	控制 LED1	1-亮
	0003H	DO	控制 LED2	1-亮
	0004H	DO	控制 LED3	1-亮
	0000H	AI	读取温度	单位 0.1 摄氏度 16 位有符号整数
	0001H	AI	读取湿度	单位 0.1%RH 16 位有符合整数

串口参数为：波特率 115200,8 个数据位，1 个停止位，无检验位。

2．要实现的功能

这个程序的用法为：

```
sensor_test  /dev/ttymxc2  beep1 on   /* 让蜂鸣器1发出声音，beep1可以换位beep2 */
sensor_test  /dev/ttymxc2  beep1 off  /* 让蜂鸣器1停止发出声音，beep1可以换位beep2 */
sensor_test  /dev/ttymxc2  led1 on    /* 点亮LED1，可以换为LED2、LED3 */
sensor_test  /dev/ttymxc2  led1 off   /* 熄灭LED1，可以换为LED2、LED3 */
sensor_test  /dev/ttymxc2  temp       /* 读取温度 */
sensor_test  /dev/ttymxc2  humi       /* 读取湿度 */
```

3．让 AI 编写程序

提示词如下：

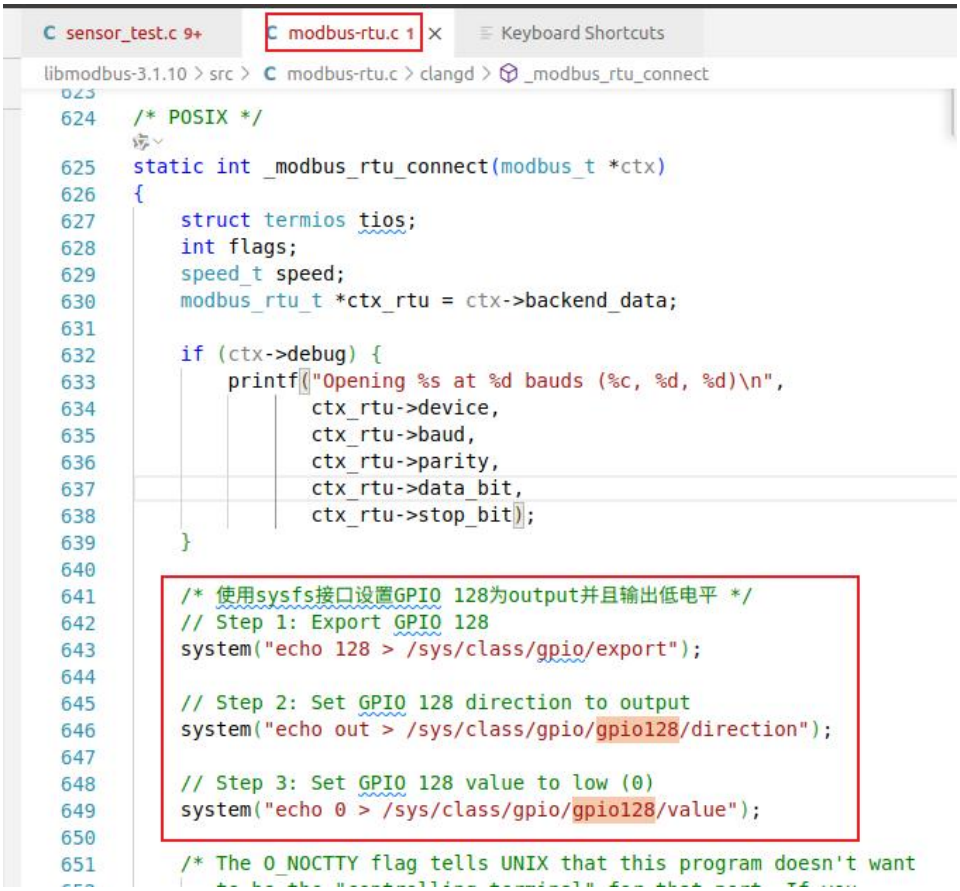
```
生成sensor_test.c，实现如下功能：
sensor_test /dev/ttymxc2 beep1 on  /* 让蜂鸣器1发出声音，beep1可以换位beep2 */
sensor_test /dev/ttymxc2 beep1 off /* 让蜂鸣器1停止发出声音，beep1可以换位beep2 */
sensor_test /dev/ttymxc2 led1 on   /* 点亮LED1，可以换为LED2、LED3 */
sensor_test /dev/ttymxc2 led1 off  /* 熄灭LED1，可以换为LED2、LED3 */
sensor_test /dev/ttymxc2 temp      /* 读取温度 */
sensor_test /dev/ttymxc2 humi      /* 读取湿度 */
其中，beep1、beep2对应DO寄存器0、1
led1、led2、led3对应DO寄存器2、3、4
temp对应AI寄存器0，
humi对应AI寄存器1
```

编译命令：

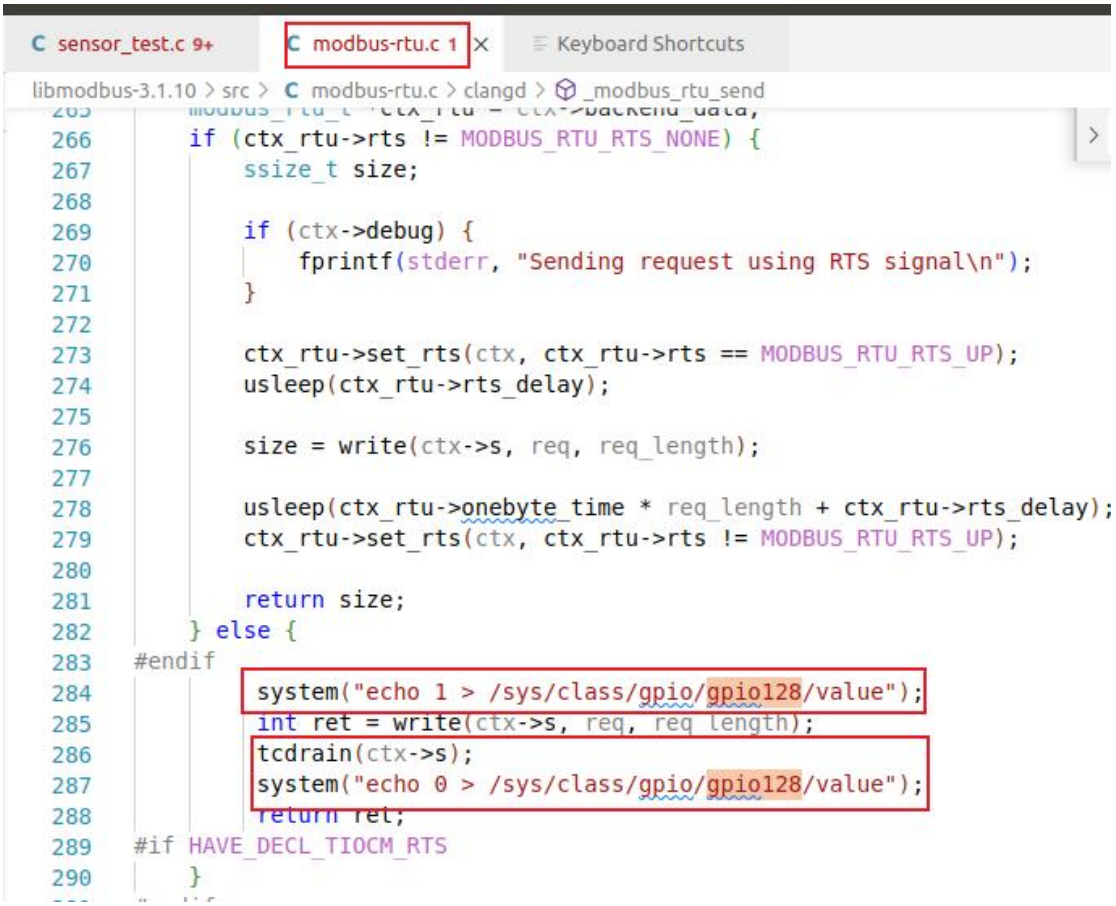
```
arm-buildroot-linux-gnueabihf-gcc -I /home/book/sensor_test/libmodbus-
3.1.10/tmp/include/modbus -L/home/book/sensor_test/libmodbus-3.1.10/tmp/lib -lmodbus  -o
sensor_test sensor_test.c
```

4．添加 RS485 的 GPIO 控制

初始化 GPIO：

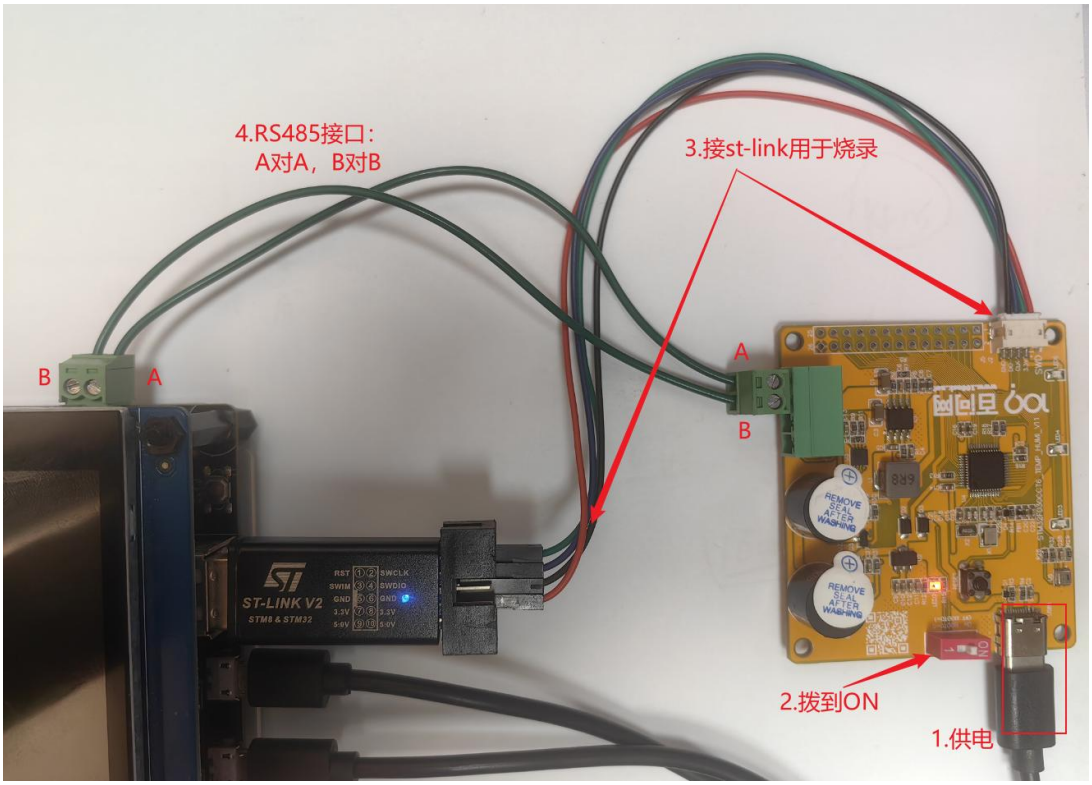


发送数据前设置 GPIO 为 1，发送完后设置 GPIO 为 0：


sensor_test.c 9+
modbus-rtu.c 1
Keyboard Shortcuts
libmodbus-3.1.10 > src > modbus-rtu.c > clangd > _modbus_rtu_send
266 if (ctx_rtu->rts != MODBUS_RTU_RTS_NONE) {
267 ssize_t size;
268
269 if (ctx->debug) {
270 fprintf(stderr, "Sending request using RTS signal\n");
271 }
272
273 ctx_rtu->set_rts(ctx, ctx_rtu->rts == MODBUS_RTU_RTS_UP);
274 usleep(ctx_rtu->rts_delay);
275
276 size = write(ctx->s, req, req_length);
277
278 usleep(ctx_rtu->onebyte_time * req_length + ctx_rtu->rts_delay);
279 ctx_rtu->set_rts(ctx, ctx_rtu->rts != MODBUS_RTU_RTS_UP);
280
281 return size;
282 } else {
283 #endif
284 system("echo 1 > /sys/class/gpio/gpio128/value");
285 int ret = write(ctx->s, req, req_length);
286 tcdrain(ctx->s);
287 system("echo 0 > /sys/class/gpio/gpio128/value");
288 return ret;
289 #if HAVE_DECL_TIOCM_RTS
290 }

5.2.4 上机测试发现 Bug

1．接线



2．测试

先上传 libmodbus 库、测试程序 sensor_test、传感器 bin 文件 demo.bin，再 Ubuntu 执行命令：

```
cd /home/book/sensor_test/libmodbus-3.1.10/tmp/lib
adb push libmodbus.so* /lib
adb  push  /home/book/sensor_test/sensor_test  /bin
adb  push  /home/book/sensor_test/demo.bin  /root
```

再在开发板烧写传感器程序,烧写命令为：

```
cd /etc/openocd/scripts

openocd -f interface/stlink.cfg -f target/stm32f0x.cfg -c init -c halt -c "flash write_image erase /root/demo.bin 0x08000000" -c reset -c shutdown

openocd -f interface/stlink.cfg -f target/stm32f0x.cfg -c init -c halt -c "flash write_image erase /root/demo_delay100ms.bin 0x08000000" -c reset -c shutdown
```

最后在开发板执行测试命令：

```
sensor_test /dev/ttymxc2 beep1 on  /* 让蜂鸣器1发出声音，beep1可以换位beep2 */
sensor_test /dev/ttymxc2 beep1 off /* 让蜂鸣器1停止发出声音，beep1可以换位beep2 */
```

```
sensor_test /dev/ttymxc2 led1 on   /* 点亮LED1，可以换为LED2、LED3 */
sensor_test /dev/ttymxc2 led1 off  /* 熄灭LED1，可以换为LED2、LED3 */
sensor_test /dev/ttymxc2 temp      /* 读取温度 */
sensor_test /dev/ttymxc2 humi      /* 读取湿度 */
```

测试时发现无法得到回应：

```
sh-5.0#
sh-5.0#
sh-5.0# sensor_test /dev/ttymxc2 led1 off
Opening /dev/ttymxc2 at 115200 bauds (N, 8, 1)
sh: line 0: echo: write error: Device or resource busy
[03][05][00][02][00][00][6D][E8]
Waiting for a confirmation...
ERROR Connection timed out: select
Connection timed out
sh-5.0#
```

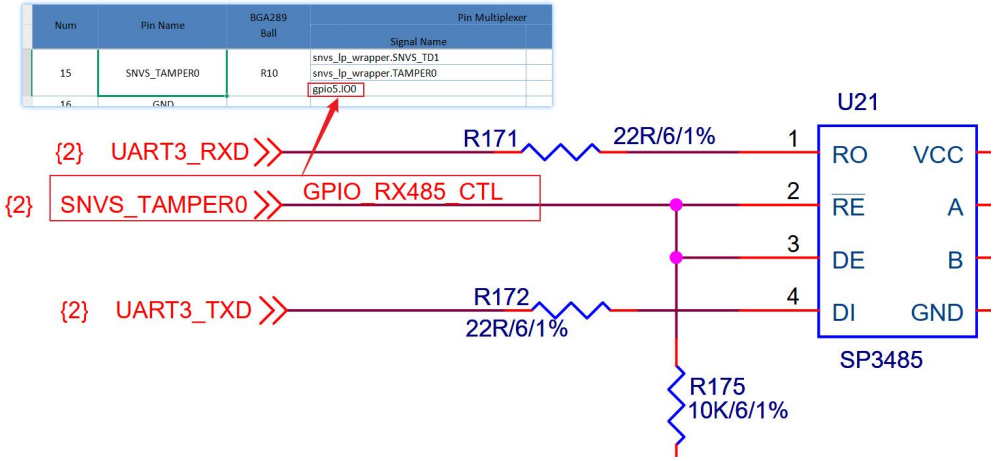
3．原因



5.3 IMX6ULL 下 RS485 接口的使用

5.3.1 设备树修改与 APP 编程

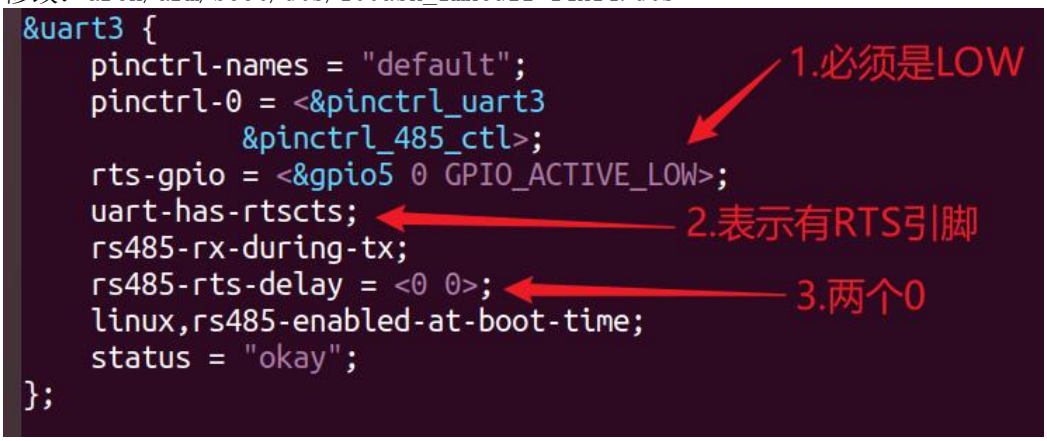
对于如下 RS485 接口，在应用程序里修改 GPIO 的状态来控制 RS485 的话，应用程序可能由于系统调度的原因无法快速运行，会使得 GPIO 引脚的修改不及时，导致 RS485 无法快速进入接收状态，从而丢失数据。



解决方法是修改设备数，在内核里启动 RS485 的引脚控制功能。

1．修改设备树

修改：arch/arm/boot/dts/100ask_imx6ull-14x14.dts



设备树的说明：

① “rts-gpio = <&gpio5 0 GPIO_ACTIVE_LOW>;”

指定 RTS 引脚，它是 RS485 的一个输出引脚，处于 ACTIVE 状态时输出低电平，处于 INACTIVE 状态时输出高电平。

② “uart-has-rtscts;”：表示有 RTS 引脚

③ “rs485-rts-delay = <0 0>;”

第 1 个值表示“改变 RTS 引脚后，再过多少 ms 才发送数据”；

第 2 个值表示“发送完最后一个数据后，再过多少 ms 才改变 RTS 引脚”。

2．使用 RS485 接口

发送数据前后，如何让驱动程序设置 RTS 引脚呢？

示例代码如下：

```
#include <linux/serial.h>
```

```
/* 用到这2个ioctl: TIOCGRS485, TIOCSRS485 */
#include <sys/ioctl.h>

struct serial_rs485 rs485conf;

/* 打开串口设备 */
int fd = open ("/dev/mydevice", O_RDWR);
if (fd < 0) {
        /* 失败则返回 */
    return -1;
}

/* 读取rs485conf */
if (ioctl (fd, TIOCGRS485, &rs485conf) < 0) {
        /* 处理错误 */
}

/* 使能RS485模式 */
rs485conf.flags |= SER_RS485_ENABLED;

/* 当发送数据时, RTS为INACTIVE状态 */
rs485conf.flags |= SER_RS485_RTS_ON_SEND;

/* 或者: 当发送数据时, RTS为ACTIVE状态 */
rs485conf.flags &= ~(SER_RS485_RTS_ON_SEND);

/* 当发送完数据后, RTS为INACTIVE状态 */
rs485conf.flags |= SER_RS485_RTS_AFTER_SEND;

/* 或者: 当发送完数据后, RTS为ACTIVE状态 */
rs485conf.flags &= ~(SER_RS485_RTS_AFTER_SEND);

/* 还可以设置:
 * 发送数据之前先设置RTS信号, 等待一会再发送数据
 * 等多久? delay_rts_before_send(单位ms)
 */
rs485conf.delay_rts_before_send = ...;

/* 还可以设置:
 * 发送数据之后, 等待一会再清除RTS信号
 * 等多久? delay_rts_after_send(单位ms)
 */
rs485conf.delay_rts_after_send = ...;
```

```
/* 如果想在发送RS485数据的同时也接收数据，还可以这样设置 */
rs485conf.flags |= SER_RS485_RX_DURING_TX;

if (ioctl (fd, TIOCSRS485, &rs485conf) < 0) {
        /* 处理错误 */
}

/* 使用read()和write()就可以读、写数据了 */

/* 关闭设备 */
if (close (fd) < 0) {
        /* 处理错误 */
}
```

简单总结就是，需要设置 rs485conf.flags，使能 RS485、选择发送数据前后是 ACTIVE 还是 INACTIVE RTS 引脚。

ACTIVE、INACTIVE 是一个逻辑值，表示 RTS 引脚输出逻辑 1、0。逻辑 1、0 对应高电平还是低电平，这取决于设备树里引脚的描述：“rts-gpio = <&gpio5 0 GPIO_ACTIVE_LOW>;”。

可以得到下表：

rs485conf.flags	设备树 RTS 引脚	说明
设置 SER_RS485_RTS_ON_SEND	GPIO_ACTIVE_LOW	发送数据时 INACTIVE，输出高电平
	GPIO_ACTIVE_HIGH	发送数据时 INACTIVE，输出低电平
清除 SER_RS485_RTS_ON_SEND	GPIO_ACTIVE_LOW	发送数据时 ACTIVE，输出低电平
	GPIO_ACTIVE_HIGH	发送数据时 ACTIVE，输出高电平
设置 SER_RS485_RTS_AFTER_SEND	GPIO_ACTIVE_LOW	发送完数据后 INACTIVE，输出高电平
	GPIO_ACTIVE_HIGH	发送完数据后 INACTIVE，输出低电平
清除 SER_RS485_RTS_AFTER_SEND	GPIO_ACTIVE_LOW	发送完数据后 ACTIVE，输出低电平
	GPIO_ACTIVE_HIGH	发送完数据后 ACTIVE，输出高电平

设备树里，引脚是 GPIO_ACTIVE_LOW，我们想在发送数据时引脚为高电平，发送完数据后引脚为低电平，就需要设置 rs485conf.flags 为如下值（它们是默认取值，所以只需要使能 RS485 即可）

① 设置 SER_RS485_RTS_ON_SEND

② 清楚 SER_RS485_RTS_AFTER_SEND

代码如下：

```
struct serial_rs485 rs485conf;

/* 打开串口设备 */
int fd = open ("/dev/mydevice", O_RDWR);
if (fd < 0) {
        /* 失败则返回 */
    return -1;
}

/* 读取rs485conf */
if (ioctl (fd, TIOCGRS485, &rs485conf) < 0) {
```

```
        /* 处理错误 */
}

/* 使能RS485模式 */
rs485conf.flags |= SER_RS485_ENABLED;
if (ioctl (fd, TIOCSRS485, &rs485conf) < 0) {
        /* 处理错误 */
}

/* 使用read()和write()就可以读、写数据了 */

/* 关闭设备 */
if (close (fd) < 0) {
        /* 处理错误 */
}
```

5.3.2 修改程序与上机实验

1．设备树

修改、编译、替换设备树，在 Ubuntu 上修改设备树后执行如下命令：

```
cd  /home/book/100ask_imx6ull-sdk/Linux-4.9.88
make dtbs
adb push arch/arm/boot/dts/100ask_imx6ull-14x14.dtb /boot
```

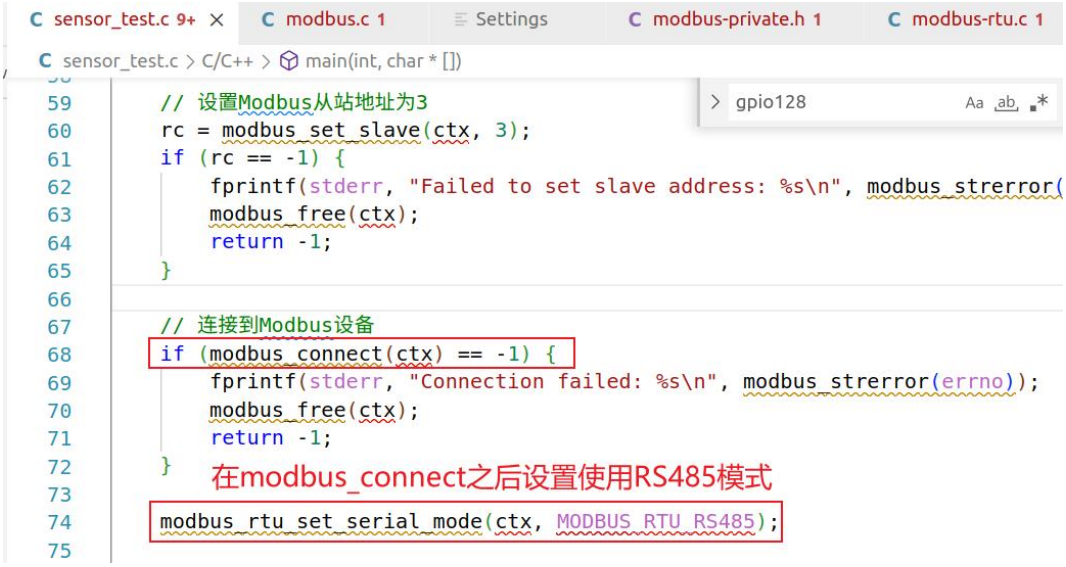
2．libmodbus

恢复 libmodbus 库：重新解压、配置、编译、上传，在 Ubuntu 上执行如下命令：

```
tar xzf libmodbus-3.1.10.tar.gz
cd libmodbus-3.1.10/
./configure --host=arm-buildroot-linux-gnueabihf --prefix=$PWD/tmp
make
make install
cd tmp/lib
adb push libmodbus.so* /lib
```

3．测试程序

修改测试程序，使用 RS485 功能，如下修改：



使用如下命令编译、上传：

```
arm-buildroot-linux-gnueabihf-gcc -I /home/book/sensor_test/libmodbus-
3.1.10/tmp/include/modbus -L/home/book/sensor_test/libmodbus-3.1.10/tmp/lib -lmodbus  -o
sensor_test sensor_test.c

adb  push  sensor_test  /bin
```

4．上机测试

在开发板烧写传感器程序,烧写命令为：

```
cd /etc/openocd/scripts

openocd -f interface/stlink.cfg -f target/stm32f0x.cfg -c init -c halt -c "flash write_image
erase /root/demo.bin 0x08000000" -c reset -c shutdown
```

最后在开发板执行测试命令：

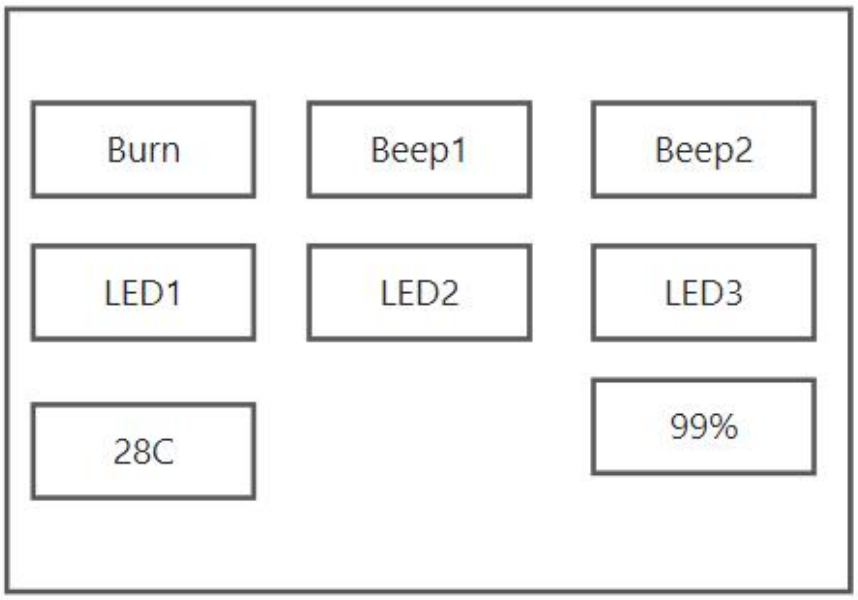
```
sensor_test /dev/ttymxc2 beep1 on  /* 让蜂鸣器1发出声音，beep1可以换位beep2 */
sensor_test /dev/ttymxc2 beep1 off /* 让蜂鸣器1停止发出声音，beep1可以换位beep2 */
sensor_test /dev/ttymxc2 led1 on   /* 点亮LED1，可以换为LED2、LED3 */
sensor_test /dev/ttymxc2 led1 off  /* 熄灭LED1，可以换为LED2、LED3 */
sensor_test /dev/ttymxc2 temp      /* 读取温度 */
sensor_test /dev/ttymxc2 humi      /* 读取湿度 */
```

第 6 章　集成 QT+OpenOCD+Modbus

6.1 效果与方案

6.1.1 效果

界面如下：点击 Burn 进行烧写，点击 Beep1~Beep2 操作蜂鸣器，点击 LED1~LED3 操作 LED，自动显示温度、湿度。



6.1.2 方案

点击按钮时，使用 system 系统调用执行命令。
QT 里启动定时器，使用 system 系统调用得到温湿度值，更新界面。

6.2 编写 QT 集成程序

6.3 编写脚本

在 kimi.com 上，使用如下提示词生成脚本：

```
生成脚本 sensor_test.sh

用法如下：
/bin/sensor_test.sh led1 on
/bin/sensor_test.sh led1 off
/bin/sensor_test.sh beep1 on
/bin/sensor_test.sh beep2 on
/bin/sensor_test.sh humi
/bin/sensor_test.sh temp
```

```
里面会分别调用：
/bin/sensor_test  /dev/ttymxc2 led1 on
/bin/sensor_test  /dev/ttymxc2 led1 on
/bin/sensor_test  /dev/ttymxc2 led1 off
/bin/sensor_test  /dev/ttymxc2 beep1 on
/bin/sensor_test  /dev/ttymxc2 beep2 on
/bin/sensor_test /dev/ttymxc2 humi
/bin/sensor_test /dev/ttymxc2 temp

当执行如下命令时，可以得到28.5  99.1这样的字符串，返回给被调用者
/bin/sensor_test.sh humi
/bin/sensor_test.sh temp
```

然后再次使用以下提示词修改脚本：

```
result数据格式为：
Opening /dev/ttymxc2 at 115200 bauds (N, 8, 1)
[03][04][00][00][00][01][30][28]
Waiting for a confirmation...
<03><04><02><01><5B><80><9B>
Temperature: 34.7 C
或：
Opening /dev/ttymxc2 at 115200 bauds (N, 8, 1)
[03][04][00][01][00][01][61][E8]
Waiting for a confirmation...
<03><04><02><01><86><40><C2>
Humidity: 39.0 %
修改脚本，挑出里面的34.7 或 39.0 作为 结果打印出来
```

6.4 上机实验

把脚本文件放入开发板/bin 目录：

```
/bin/sensor_test.sh
/bin/burn.sh
```

确保开发板有如下文件或目录：

```
/etc/openocd 目录
/root/demo.bin
/bin/sensor_test
/bin/openocd
/bin/iot_sensor_test
```

在开发部上执行命令：

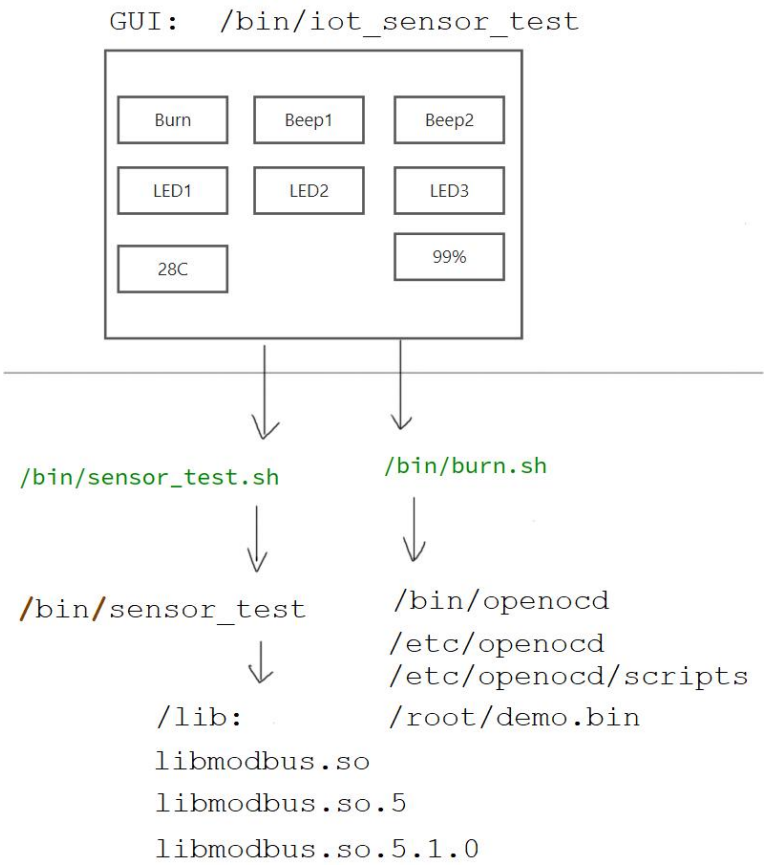
```
echo -e "\033[9;0]" > /dev/tty0
export QT_QPA_GENERIC_PLUGINS=tslib:/dev/input/event1
export QT_QPA_PLATFORM=linuxfb:fb=/dev/fb0
export QT_QPA_FONTDIR=/usr/lib/fonts/
```

```
/bin/iot_sensor_test
```

第 7 章　制作镜像文件

7.1 准备所有 APP 和脚本

7.1.1 程序相关的文件



制作好的 APP 和脚本放在这里：



执行一下命令放到板子上即可：

```
tar xjf iot_sensor_test_output.tar.bz2
cd iot_sensor_test_output/
adb push * /
```

然后重启板子。

7.1.2 启动相关的文件

/boot/100ask_imx6ull-14x14.dtb、/etc/init.d/S99myqt

创建一个文件：/etc/init.d/S99myqt，内容如下：

```
#!/bin/sh
```

```
start() {
echo -e "\033[9;0]" > /dev/tty0
export QT_QPA_GENERIC_PLUGINS=tslib:/dev/input/event1
export QT_QPA_PLATFORM=linuxfb:fb=/dev/fb0
export QT_QPA_FONTDIR=/usr/lib/fonts/

psplash-write "PROGRESS 95"
psplash-write "QUIT"
/bin/iot_sensor_test  &
}

stop() {
    killall  iot_sensor_test
}

case "$1" in
    start)
        start
        ;;
    stop)
        stop
        ;;
    *)
        echo "Usage: $0 {start| stop|restart}"
        exit 1
esac

exit $?
```

7.2 修改镜像文件

有些板卡只有镜像文件没有源码，我们无法自己编译制作。有些镜像文件的编译过程麻烦、耗时，而我们只想简单修改。

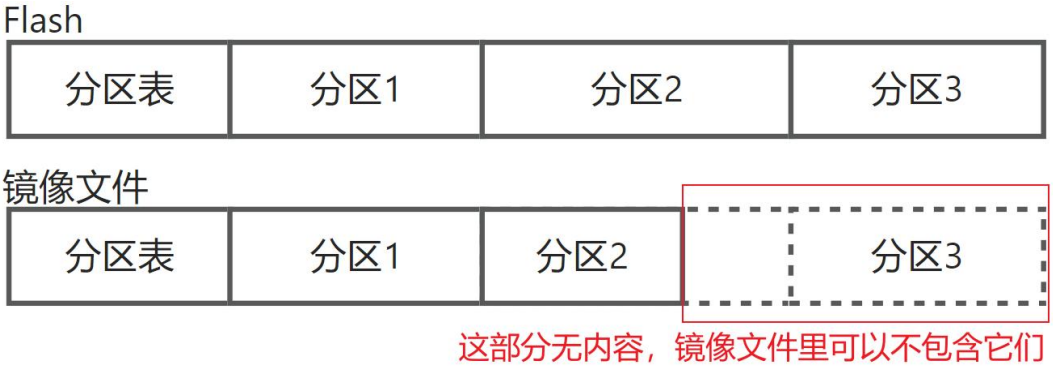
怎么办？

① 烧录进板子后，启动板子，添加自己的文件：无法批量生产，操作麻烦

② 暴力修改镜像文件，以后烧写镜像文件即可：可以批量生产，操作简单

7.2.1 Flash 与镜像文件的关系

Flash 上有分区表、各个分区，容量可能很大。而镜像文件并不一定跟 Flash 一样大，如下图所示：



在镜像文件里，本该含有分区表、分区 1~分区 3 的所有数据。如果制作镜像文件时，分区 2 没有使用完，分区 3 无有效数据，那么镜像文件中：

① 对应分区 2 的数据可以比分区 2 小

② 可以完全不包含分区 3 的数据。

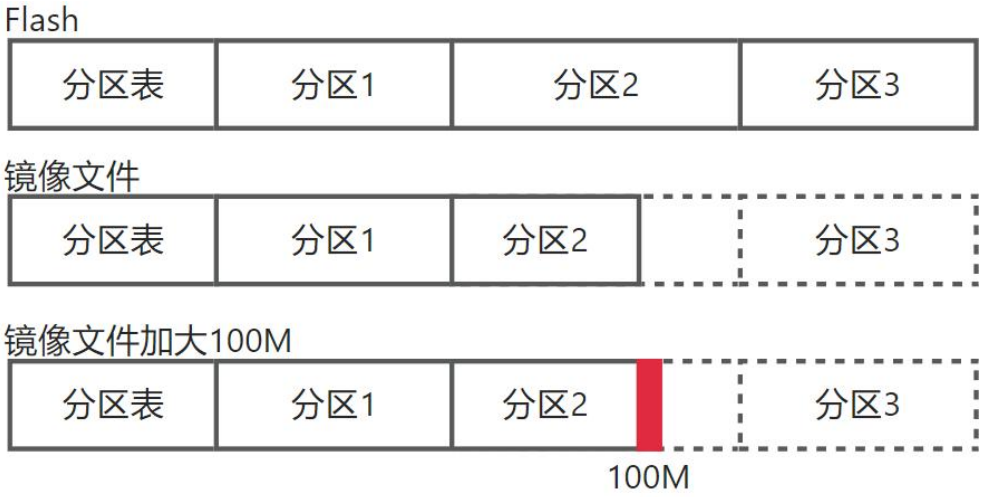
7.2.2 修改镜像文件

1．扩容

从上图可以看到镜像文件里“一丝多余空间都没有”，要往里面加东西，需要扩大它的容量。比如使用下面命令加大 100M：

```
truncate -s +100M emmc.img
```

镜像文件变化如下：



2．挂载

使用 losetup 把 emmc.img 设置为回环设备：

```
sudo losetup -P /dev/loop30 emmc.img
$ ls /dev/loop30* -1
/dev/loop30      // 对应整个映像文件
/dev/loop30p1    // 对应第1个分区
/dev/loop30p2    // 对应第2个分区
```

挂载、更新大小：

```
sudo  mount  /dev/loop30p2   /mnt
sudo  resize2fs  /dev/loop30p2
```

3．加入文件、修改文件

在“/mnt”目录下就可以加入文件，修改文件了，比如：

```
sudo cp ~/create_fs/iot_sensor_test_output/* /mnt  -rfd
```

修改启动脚本，参考《7.2.3 修改启动脚本》。

4．退出

```
sudo  unmount   /mnt
sudo  losetup  -d  /dev/loop30
```

7.2.3 修改启动脚本

系统启动后，第 1 个进程被称为 init 进程，它根据配置文件“/etc/inittab”启动其他程序。

“/etc/inittab”内容如下：

```
book@100ask:/mnt$ cat etc/inittab
# /etc/inittab
#
# Copyright (C) 2001 Erik Andersen <andersen@codepoet.org>
#
# Note: BusyBox init doesn't support runlevels.  The runlevels field is
# completely ignored by BusyBox init. If you want runlevels, use
# sysvinit.
#
# Format for each entry: <id>:<runlevels>:<action>:<process>
#
# id        == tty to run on, or empty for /dev/console
# runlevels == ignored
# action    == one of sysinit, respawn, askfirst, wait, and once
# process   == program to run

# Startup the system
::sysinit:/bin/mount -t proc proc /proc
::sysinit:/bin/mount -o remount,rw /
::sysinit:/bin/mkdir -p /dev/pts /dev/shm
::sysinit:/bin/mount -a
::sysinit:/sbin/swapon -a
null::sysinit:/bin/ln -sf /proc/self/fd /dev/fd
null::sysinit:/bin/ln -sf /proc/self/fd/0 /dev/stdin
null::sysinit:/bin/ln -sf /proc/self/fd/1 /dev/stdout
null::sysinit:/bin/ln -sf /proc/self/fd/2 /dev/stderr
::sysinit:/bin/hostname -F /etc/hostname
# now run any rc scripts
::sysinit:/etc/init.d/rcS

# Put a getty on the serial port
ttymxc0::respawn:/sbin/getty -L  ttymxc0 0 vt100 # GENERIC_SERIAL

# Stuff to do for the 3-finger salute
#::ctrlaltdel:/sbin/reboot
```

1.挂载文件系统

2.运行脚本

3.运行登录程序

“/etc/init.d/rcS”脚本内容如下：

```
book@100ask:/mnt$ cat etc/init.d/rcS
#!/bin/sh


# Start all init scripts in /etc/init.d
# executing them in numerical order.
#
psplash -n &
for i in /etc/init.d/S??* ;do

     # Ignore dangling symlinks (if any).
     [ ! -f "$i" ] && continue

     case "$i" in
        *.sh)
            # Source shell script for speed.
            (
                trap - INT QUIT TSTP
                set start
                . $i
            )
            ;;
        *)
            # No sh extension, so fork subprocess.
            $i start
            ;;
    esac
done
```

1.对于/etc/init.d下以S开头的文件

2.如果是脚本，用source命令运行它，用来设置环境变量等

3.如果不是脚本，运行它的start函数

对于 GUI 启动，我们只能保留一个：

```
book@100ask:/mnt$ ls etc/init.d/
bluetooth  S01syslogd  S05lvgl     S20urandom  S40network          S49ntp         S50sshd     S98swupdate  S99myqt
rcK        S02klogd    S09modload  S30dbus     S44modem-manager    S50mosquitto   S50telnet   S99adbd
rcS        S02sysctl   S10udev     S35resize   S45network-manager  S50pulseaudio  S80dnsmasq  S99myirhmi2
book@100ask:/mnt$
```